

Évolution des réseaux booléens à signe

Ai-Ling Nguyen Bonnet, encadrée par Sergiu Ivanov et Nicolas Glade

February 21, 2023

Contents

1	Introduction	3
1.1	Les réseaux booléens, à la mano	3
1.2	Les SBNs pour parler d'évolution de la vie - rien que ça	3
2	Présentation des SBN	3
2.1	Structure des SBN	3
2.1.1	Quelques exemples pour se l'imaginer	4
2.1.2	SBF et graphe d'interaction	5
2.2	Dynamique des SBNs	5
2.3	Réseaux et classes d'équivalence	6
3	Évolution de SBNs	7
3.1	Concepts évolutifs	7
3.1.1	Robustesse et évolutivité	7
3.1.2	Viabilité	8
3.2	Distances sur les SBNs	10
3.2.1	Distances structurelles	10
3.2.2	Distances dynamiques	12
3.2.3	Structure de l'espace des réseaux	15
3.3	Formalisation de l'évolution	16
3.3.1	Robustesse	16
3.3.2	Coût d'un chemin d'évolution, coût d'une évolution	16
3.3.3	Évolutivité	17
3.3.4	Viabilité et score de viabilité	18
3.4	Paysage des réseaux	19
3.4.1	Métagraphe des SBNs	19
3.4.2	Reformulation de certains concepts avec les métagraphes	20
3.5	Synthèse	22
3.5.1	Tableau de synthèse sur les distances	22
3.5.2	Evolution	23

4	Developpement logiciel	24
4.1	Calculs en Java	24
4.1.1	Les classes d'objets	24
4.1.2	La librairie JPP	25
4.1.3	Exploration exhaustive, en d2 et d3	26
4.2	Analyse de données en Python	26
4.2.1	Generation de heatmaps	27
4.2.2	Correlation	27
4.2.3	Metagraphe	28
4.2.4	Robustesse et évolutivité	28
4.2.5	Trouver un chemin	28
5	Résultats et analyse	30
5.1	Listes de SBNs et distances	30
5.2	Les heatmaps	30
5.3	Correlations	32
5.4	Metagraphe	33
5.5	Robustesse	36
5.6	Evolutivité	39
6	Conclusion	41
7	Quelques conventions de travail	42
7.1	Ecriture binaire	42
7.2	Dénominations	42
7.2.1	Pour ne pas se faire de noeuds au cerveau... Désignation des noeuds	42
7.2.2	Sigle des distances	42
7.2.3	Nom des fichiers	43
7.2.4	Indexation des SBNs	43
7.3	Mémo	43
7.4	Classe d'équivalence de SBNs	43
7.5	Statistiques utiles	43

1 Introduction

1.1 Les réseaux booléens, à la mano

Les gènes d'une cellule ou les neurones d'un cerveau, finalement c'est la même chose. Ah bon?

En effet, ce sont deux systèmes biologiques dont les composantes interagissent entre elles et forment ainsi un réseau d'interactions biologiques. Plusieurs types de réseaux peuvent être utilisés pour modéliser de tels biosystèmes. Les réseaux sont des modèles formels qui peuvent représenter des systèmes vivants (ou pas d'ailleurs), présentant plusieurs composantes distinctes et en interaction les unes avec les autres. Les réseaux booléens à signe (SBN, Signed Boolean Networks) sur lesquels j'ai travaillé ont la particularité d'avoir des composantes prenant des valeurs binaires (ou pouvant être approximées comme telles). Cela permet d'avoir une structure simple à construire mais qui permet des analyses assez poussées sur le système. L'équipe COSMO dans laquelle je fais mon stage travaille sur la théorie des SBN, et non pas sur la modélisation de systèmes réels avec ce modèle. Cette formalisation permet de se placer dans un cadre conceptuel et donc très général. Si j'esquisse quelques exemples par la suite, c'est pour illustrer le concept plutôt que de modéliser concrètement et rigoureusement les exemples choisis.

1.2 Les SBNs pour parler d'évolution de la vie - rien que ça

"Comment un système biologique a-t-il évolué en un autre?" Voici une grande question de biologie que l'on tente d'aborder avec ce modèle de SBN [3]. On peut appliquer une mutation sur un SBN et donc le faire évoluer par itération. Pour de petits systèmes, on peut même dresser un paysage des réseaux. On construit tous les SBNs possibles pour regarder leur évolution en tant qu'ensemble: comment un réseau peut évoluer en un autre, à quel point les comportements sont conservés dans l'évolution ou à quel point le système peut adopter de nouveaux comportements... Ce sont tout autant de considérations qui ont guidé mon stage. La théorisation de l'évolution de SBNs a été faite par Rémy Defossez dans son rapport de stage [?]. Mon travail a surtout consisté à implémenter les concepts ainsi développés pour faire des analyses dessus. Ces analyses ont été faites sur de petits systèmes (SBNs de dimensions 2 et 3), ce qui a permis de faire des calculs exhaustifs (sur tous les SBNs d'une dimension).

Ce rapport présente d'abord les concepts théoriques, en reprenant le travail de Rémy Defossez (n'hésitez pas à lire son rapport qui sera plus complet [1]), suivi de mon travail d'implémentation et d'analyse sur ces concepts.

2 Présentation des SBN

2.1 Structure des SBN

Un SBN est un graphe booléen orienté pondéré. Sa **structure** est donnée par:

- Ses noeuds qui sont booléens, *ie* à valeur dans $\{0; 1\}$.
- Ses arcs orientés d'un noeud de départ i vers un noeud d'arrivée j , et qui ont un poids dans $\mathbb{Z}$. Un poids positif représente un effet "activateur" du noeud i sur j ; un poids négatif caractérise un effet "inhibiteur".

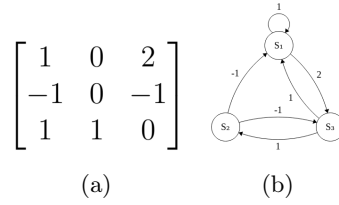


Figure 1

Comme tout graphe, un SBN peut aussi être représenté par sa matrice d'adjacence et un schéma des noeuds et arcs.

On notera qu'il s'agit de graphes déterministes et non probabilistes.

2.1.1 Quelques exemples pour se l'imaginer

Tachons d'appliquer notre construction à des exemples simples mais très généraux. Ces exemples permettent de voir que les SBN s'appliquent à des systèmes biologiques très divers, à tous les niveaux d'organisation du vivant. Cependant, ce ne sont que des esquisses dont la modélisation n'est pas entièrement rigoureuse et présente des failles. L'idée est juste de donner corps aux concepts.

Réseau génétique Soit un réseau génétique à d gènes qui se régulent entre eux.

- Noeud i : Un gène avec deux niveaux d'expression: activé (1) ou désactivé (0)
- Arcs: Les forces de régulation intergènes. Un arc de poids positif de i vers j représente une régulation activatrice du gène i sur le gène j (qui permet au gène j de s'exprimer). Au contraire, un arc négatif représente une régulation inhibitrice, qui empêche j de s'exprimer. La valeur même du poids représente la force de la régulation (plus ou moins forte). Ainsi, on peut représenter le fait qu'un gène puisse être régulé par plusieurs autres gènes, dont lui-même. L'activation ou l'inhibition de ce gène va dépendre de la régulation globale.

Ecosystème On peut choisir les noeuds comme étant les espèces de l'écosystème, les arcs les interactions interspécifiques qui agissent comme des régulateurs

- Noeud i : l'espèce i de l'écosystème, en terme d'abondance. On considère que cette espèce ne peut atteindre que deux états d'abondance: un état minimal A_{imin} , mettant le noeud à 0, et un état maximal A_{imax} mettant le noeud à 1. Les noeuds auront tous une valeur booléenne 0 ou 1, mais les abondances en tant que telles sont propres à chaque espèce. La valeur du noeud est donnée par $\frac{A-A_{imin}}{A_{imax}-A_{imin}}$ (avec $A \in \{A_{imin}, A_{imax}\}$)
- Arcs: interactions interspécifiques, qui agissent comme des leviers de régulation. Ainsi la prédation est une action inhibitrice car elle fait diminuer la population (poids négatif), le mutualisme ou la symbiose une action activatrice en permettant à la population de croître (poids positif).

2.1.2 SBF et graphe d'interaction

Voici une formalisation un peu plus poussée de la structure des SBNs, utilisée par l'équipe. On définit une SBF (*Signed Boolean Function*) comme étant un d -uplet f d'entiers (i.e. $f \in \mathbb{Z}^d$). On définit un état comme un d -uplet de booléens. On notera ici $\mathbb{S} = \{0, 1\}^d$ l'ensemble des états possibles.

On définit l'image d'un état s par la SBF f ainsi :

$$f(s) = \begin{cases} 1 & \text{si } \sum_{i=1}^d f_i s_i > 0 \\ 0 & \text{sinon.} \end{cases} \quad (1)$$

Un SBN de dimension d est un d -uplet de SBF reliées entre elles par des arcs associant SBF et entrées des SBF, i.e. une matrice carrée de taille d à coefficients entiers. On note donc $\mathbb{G} = \mathcal{M}_d(\mathbb{Z})$ l'ensemble des SBN. La matrice correspondante est la matrice de poids du graphe d'interaction, où g_{ij} est l'arc allant du noeud j au noeud i .

Mentionnons ici un débat encore en cours sur le rôle des arcs de poids nuls. La question est de savoir s'il y a un sens à considérer un réseau ayant de tels arcs. Si en pratique un arc de poids nul n'a jamais d'influence, il est intéressant de noter que dans la structure d'un système biologique deux éléments reliés par une liaison de force nulle ne sont pas identiques à deux éléments non liés, car cette liaison nulle peut ou bien traduire un résidu de l'évolution (la liaison a perdu en force au cours de la lignée jusqu'à être nulle) ou un potentiel évolutif. Dans le cas de notre modèle, nous choisirons cependant de ne pas distinguer arc inexistant d'un arc de poids nul, ou plutôt de considérer les SBN comme des graphes complets dont on ne représente que les arcs de poids non nul dans la représentation structurelle (figure 2(b)). Notamment, le graphe est toujours connexe, pourvu que l'on s'autorise à passer par des arcs de poids nul.

2.2 Dynamique des SBNs

Un SBN n'est pas fixe dans le temps. A chaque "instant", chaque sommet prend une valeur binaire, qui sera modifiée à l'instant suivant avec les interactions avec les autres sommets. Cette activation/désactivation des sommets est modélisée par la fonction de succession. On associe à chaque SBN g sa fonction de succession $succ_g$ définie ainsi :

$$\begin{aligned} succ_g : \mathbb{S} &\rightarrow \mathbb{S} \\ s &\mapsto (g_i(s))_{1 \leq i \leq d} \end{aligned} \quad (2)$$

Où g_i est la i -ème SBF de g , soit la i -ème ligne de la matrice de poids.

La fonction de succession correspond à ce que l'on appelle le *comportement* du système biologique. On représente la fonction de succession avec un graphe de transition comportant tous les états possibles. Un exemple est donné en figure 2(c). On comprend aisément que plusieurs SBN peuvent avoir le même comportement, car l'ensemble des réseaux est infini tandis qu'il n'y a qu'un nombre fini de comportements.

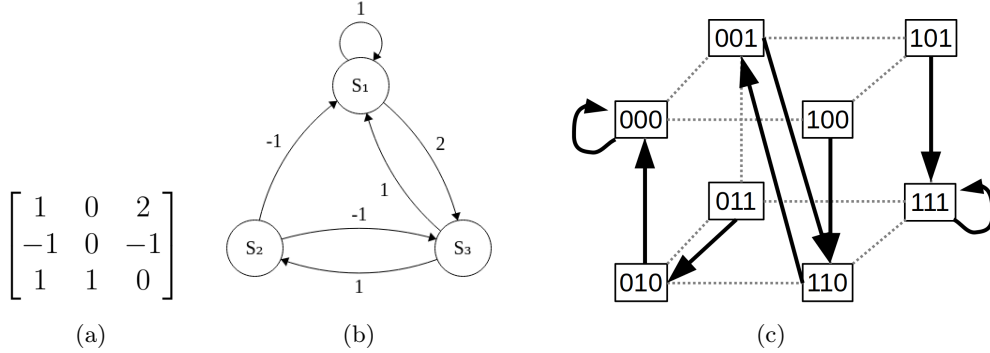


Figure 2: Trois représentations d'un même SBN. (a) : Matrice de poids ; (b) : Graphe d'interaction, représente la structure du réseau ; (c) : Graphe de transition, représente la dynamique du réseau

2.3 Réseaux et classes d'équivalence

L'un des premiers objectifs de l'équipe BCM en étudiant les SBN a été de chercher une organisation de l'espace des réseaux, et de regrouper ces réseaux par catégories, ce qui a mené naturellement à définir des classes d'équivalence.

Cette approche s'est révélée très utile en pratique dans l'étude informatique des SBN, car $\mathbb{G}$ est de cardinal infini. Pour se ramener à un ensemble d'étude plus petit, on choisit une relation d'équivalence $\mathcal{R}$ sur les SBN dépendant du modèle et on travaille sur $\mathbb{G}/\mathcal{R}$. La relation d'équivalence choisie par l'équipe où je travaille est la suivante :

$$g\mathcal{R}g' \iff \begin{cases} \forall i, TV(g_i) = TV(g'_i) \\ \forall i, j; g_{ij} = 0 \iff g'_{ij} = 0 \end{cases} \quad (3)$$

où $TV(g_i)$ est la table de vérité de la fonction représentée par g_i .

La première condition garantit qu'il n'y ait pas de différence au niveau logique entre les SBN d'une même classe, et surtout que deux éléments d'une même classe d'équivalence aient le même comportement. La deuxième condition vient du fait que les arcs de poids 0 ont une signification particulière sur le modèle, comme on l'a vu précédemment.

Cette relation a pour vocation de réduire la taille de l'espace des réseaux en perdant un minimum d'information. Cependant, même ainsi, on obtient un nombre trop important de classes d'équivalence (figure 3). C'est pourquoi on ne peut pas faire de calcul exhaustif sur l'espace des réseaux pour $d \geq 4$. D'autres approches sont possibles : par exemple, pour $d = 4$, l'équipe BCM étudie l'espace des réseaux avec un échantillonnage de 1/100.

Cependant, cette relation d'équivalence est toujours un choix de travail de l'équipe, même si elle est particulièrement adaptée à l'étude des SBN. Afin de conserver la flexibilité de notre modèle, j'ai décidé de travailler par la suite avec une relation d'équivalence $\mathcal{R}$ quelconque qui satisfait les deux conditions suivantes :

- $\mathbb{G}/\mathcal{R}$ est fini.
- Deux réseaux équivalents ont le même comportement.

dimension	nombre de classes
2	60
3	31199
4	3.623×10^9
5	$\approx 3.5 \times 10^{20}$

Figure 3: Nombres de classes d'équivalence des réseaux de dimensions 2, 3, 4 et 5

De cette manière, d'autres critères d'équivalence spécifiques aux systèmes étudiés peuvent être choisis sans perturber le reste de la formalisation.

3 Évolution de SBNs

Comme on l'a vu, le modèle formel des SBN est à la fois simple en termes de calculs et pourtant capable de prédictions efficaces sur des systèmes biologiques réels. On aimerait donc pouvoir étudier des concepts complexes dans ce modèle simple, tel que l'évolution biologique. Pour cela, on a retenu trois notions: la robustesse, l'évolutivité et la viabilité. Ces trois notions permettent d'étudier l'évolution d'un SBN et nécessitent de poser une certaine topologie sur l'espace des réseaux. Elles ont été sélectionnées et formalisées par R. Defossez.

3.1 Concepts évolutifs

La robustesse est la résistance d'un système aux mutations, ce qui est important pour une lignée afin de ne pas s'éteindre toute seule. L'évolutivité est la capacité d'un système à changer de comportement en peu de mutations, ce qui est une qualité nécessaire pour s'adapter à un environnement changeant. La viabilité est la contrainte exercée par l'environnement et la physique sur les systèmes. On parle de systèmes viables ou non. De façon plus nuancée, on parlera de conditions favorables à la vie de tels systèmes.

Ces trois notions représentent très bien les principaux axes d'étude de l'évolution, et leur définition est nécessaire pour des travaux futurs.

3.1.1 Robustesse et évolutivité

La capacité d'un système à évoluer dépend de deux qualités : il faut qu'il soit suffisamment stable au cours de la lignée pour que celle-ci puisse survivre, mais il doit être capable de s'adapter rapidement aux modifications de son environnement ou d'améliorer sa propre capacité de survie. Ces deux qualités, robustesse et évolutivité, sont nécessaires à la survie à long terme du système, que l'on étudie le réseau génétique d'une cellule ou un écosystème. Même si ces deux qualités semblent à priori antagonistes, il a été montré qu'elles peuvent être réconciliées [5]. Ce sont donc ces deux concepts que nous utiliserons pour qualifier le rapport d'un système à l'évolution.

Les définitions communément acceptées de la robustesse et de l'évolutivité en biologie théorique sont les suivantes :

- Un système biologique est dit robuste si son comportement est conservé après une mutation dans sa structure.

- Un système biologique est dit évolutif si des mutations dans sa structure peuvent induire une grande diversité de comportements.

On cherche ici à traduire ces définitions en langage formel. On comprend que la structure d'un système biologique correspond à la matrice d'interaction de nos SBN et que le comportement correspond au graphe de transition. En revanche, il n'est pas aussi évident de comprendre ce qu'est formellement une mutation et ce que signifie « conserver le comportement ». Ces deux notions demandent de pouvoir comparer des SBN, mais pour l'instant l'espace des réseaux n'a ni mesure de comparaison, ni mesure de similarité. Il sera donc nécessaire de définir une ou plusieurs distances sur l'espace des réseaux dans le formalisme.

3.1.2 Viabilité

Certains des systèmes biologiques possiblement représentés par nos SBN ne sont pas viables. Par exemple, un écosystème constitué uniquement de prédateurs s'attaquant les uns les autres n'est absolument pas viable, car il finit par s'éteindre tout seul. Mais cette viabilité peut être aussi due à l'environnement. Le métabolisme de l'ours polaire est particulièrement adapté à la banquise, mais dans un climat désertique comme le Sahara, il n'a aucune chance de survie. Il est nécessaire de prendre en compte pour notre formalisme l'action de l'environnement, que nous résumerons ici à la capacité de survie du système biologique dans cet environnement, et que nous appellerons la contrainte de viabilité, ou plus simplement la viabilité.

A priori, comme nous travaillons sur des réseaux formels détachés de la réalité, que ceux-ci peuvent représenter des systèmes très différents les uns des autres, et que pour un modèle, l'environnement peut varier, on ne peut pas se permettre de fixer une contrainte de viabilité universelle. Pour cette raison, le formalisme que j'exposerai ne devra pas définir ce qu'est dans l'absolu un réseau viable ou non, mais plutôt ce qu'est une contrainte de viabilité dans le cadre des SBN, et comparer les différentes viabilités possibles.

Pour offrir une mesure de comparaison entre les viabilités, introduisons la notion de chemins viables. Un chemin est une trajectoire dans l'espace des réseaux composée d'une succession de SBN proches. Il représente une évolution du réseau de départ en le réseau d'arrivée. Parmi les SBN composant la trajectoire, les uns sont viables et les autres non. On parle de chemin viable lorsque tous les points de la trajectoire sont viables (figure 4). Dans les systèmes biologiques réels, seules les évolutions empruntant des trajectoires viables sont possibles : tous les membres de la lignée doivent survivre suffisamment longtemps pour avoir un descendant. La question qui se pose pour deux réseaux et une viabilité donnée est la suivante : existe-t-il un chemin viable reliant l'un à l'autre, ou bien le premier système ne peut-il jamais devenir le second au cours de l'évolution ? Par exemple, sur la figure 5(a), le réseau A peut évoluer en le réseau B, mais dans un environnement plus contraignant comme la figure 5(b), l'évolution n'est pas possible.

De manière plus générale, pour une contrainte de viabilité donnée, toute évolution est-elle possible ou bien l'espace des réseaux est-il constitué d'une multitude de blocs viables isolés ? Pour répondre à cette question, on aimerait pouvoir définir un *score de viabilité*, qui serait élevé dans le cas où l'espace des réseaux viables est composé d'un bloc unique (figure 5(a)), et faible dans le cas où il a plutôt une forme d'archipel composé de petits blocs isolés (figure 5(b)).

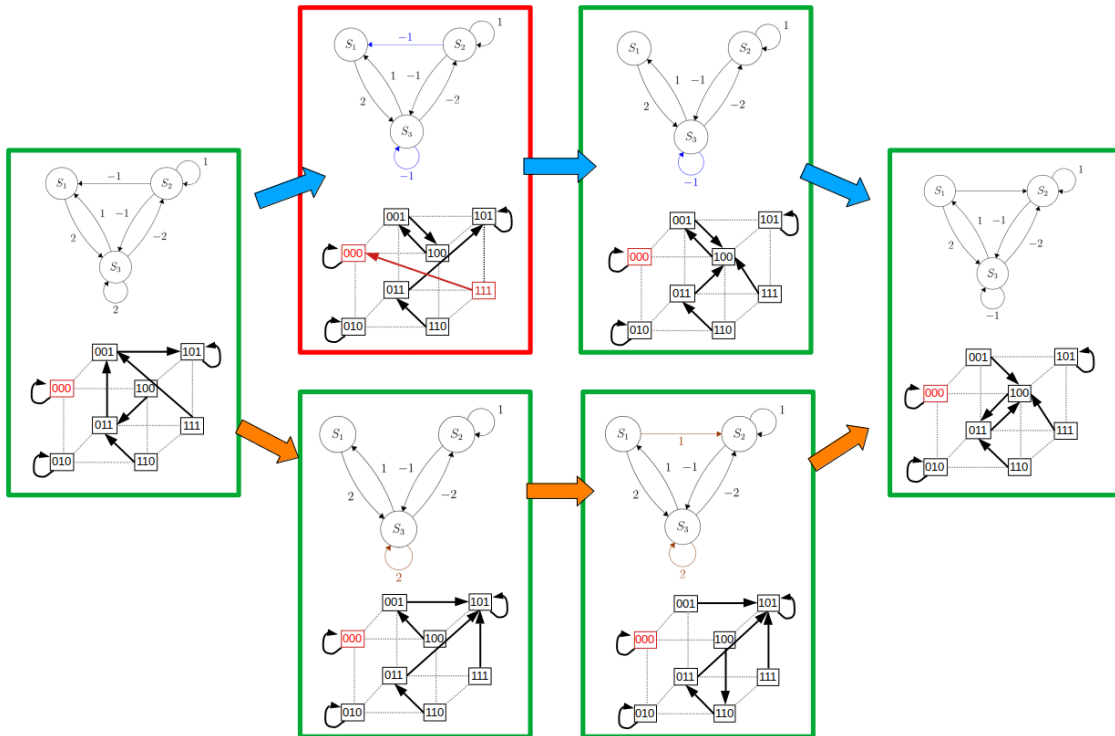


Figure 4: Deux exemples de chemins dans l'espace des réseaux. On considère ici comme voisins deux SBN qui ne diffèrent que d'un arc et comme viable un SBN dont aucun état non nul n'aboutit à l'état nul. Le chemin orange est viable, le chemin bleu ne l'est pas.

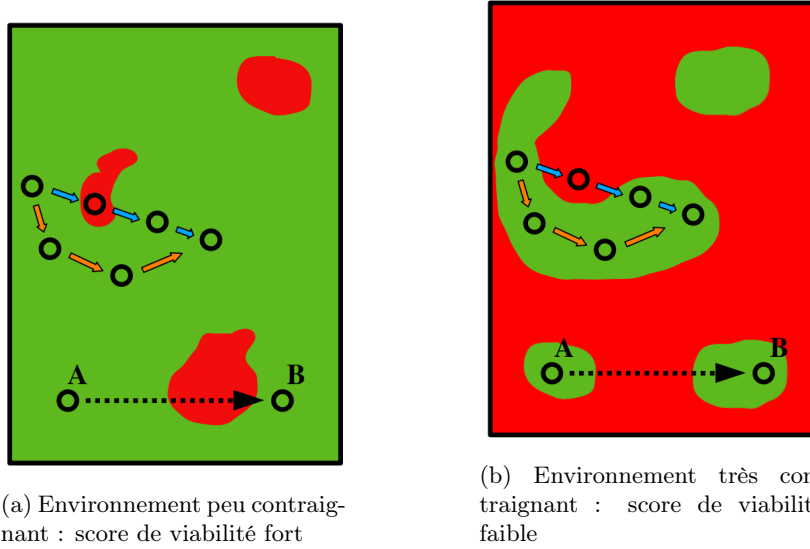


Figure 5: Représentation de l'espace des réseaux avec deux viabilités différentes : on cherche à construire un score de viabilité qui attribue une valeur plus élevée à la viabilité de gauche.

3.2 Distances sur les SBNs

Pour comparer ainsi les réseaux entre eux, il est d'abord nécessaire d'établir une certaine topologie dans l'espace des réseaux afin de pouvoir quantifier à quel points des réseaux sont "proches". Avec les SBNs, cette notion de proximité peut être étudiée sous différents angles: c'est pour cela que l'on peut définir plusieurs distances sur les réseaux. Ces distances se regroupent en deux catégories: les distances structurelles et les distances dynamiques.

3.2.1 Distances structurelles

Distance de Hamming structurelle - HSTR

A ce jour, c'est la distance sur la structure des réseaux que l'on utilise le plus, car elle a le mérite d'être simple. Elle compte le nombre de mutations nécessaires pour passer d'un réseau à un autre. Ici, une mutation est un changement de 1 sur le poids d'un arc. Elle fait donc la somme des changements pour chaque arc entre les deux réseaux.

$$\forall g, g' \in \mathbb{G}, HSTR(g, g') = \sum_{i=1}^d \sum_{j=1}^d |g_{ij} - g'_{ij}|$$

où g_{ij} est le poids de l'arc du noeud i vers le noeud j dans le graphe g .

Distance sur les chemins - PATH

La distance de Hamming qu'on a défini accorde la même importance à chacun des arcs (proportionnellement à son poids), mais on peut chercher à définir une distance qui accorde plus

d'importance à certains arcs qu'à d'autres. Par exemple, il peut exister des systèmes biologiques où l'importance d'une interaction dans le réseau impacte directement le coût de modification de cette interaction.

En théorie des graphes, définir l'importance d'un arc revient à définir ce que l'on appelle un indicateur de centralité [4]. Il n'y a pas de définition universelle de la centralité, il faut poser des critères venant du système à modéliser. Ici, on se fixera deux critères, illustrés avec la figure 6 :

- A poids équivalent, un arc a plus d'importance s'il est plus impliqué dans la connectivité du réseau.
- A position équivalente, un arc a plus d'importance si son poids (en valeur absolue) est plus élevé.

On veut construire ensuite une distance pour laquelle une modification sur un arc a un coût d'autant plus élevé que l'arc a de l'importance.

Notre nouvelle distance doit prendre en compte la connectivité du réseau : on devra faire un calcul non pas sur les arcs, mais sur les chemins qu'il contient. Ici, on ne considère que les chemins qui ne comptent pas de cycle, afin de maintenir les calculs sur des ensembles finis. En outre, on doit étendre la notion de poids aux chemins. On introduira pour cela la notion d'*influence* d'un arc et d'un chemin.

On définit l'influence d'un arc a à la position g_{ij} ainsi :

$$I_g(a) = \frac{g_{ij}}{\sum_{k=1}^d |g_{ik}|} \quad (4)$$

où g_{ij} est l'arc allant de j à i .

L'influence d'un arc représente la corrélation que l'on peut espérer entre la valeur des deux sommets liés. Si elle vaut 1, le sommet de départ détermine complètement l'état du sommet d'arrivée. Si elle est proche de zéro, alors la probabilité que la valeur du sommet de départ ait une importance pour déterminer la valeur du sommet d'arrivée est faible.

On définit ensuite l'influence d'un chemin c par la formule :

$$I_g(c) = \prod_{a \in \text{arcs}(c)} I_g(a) \quad (5)$$

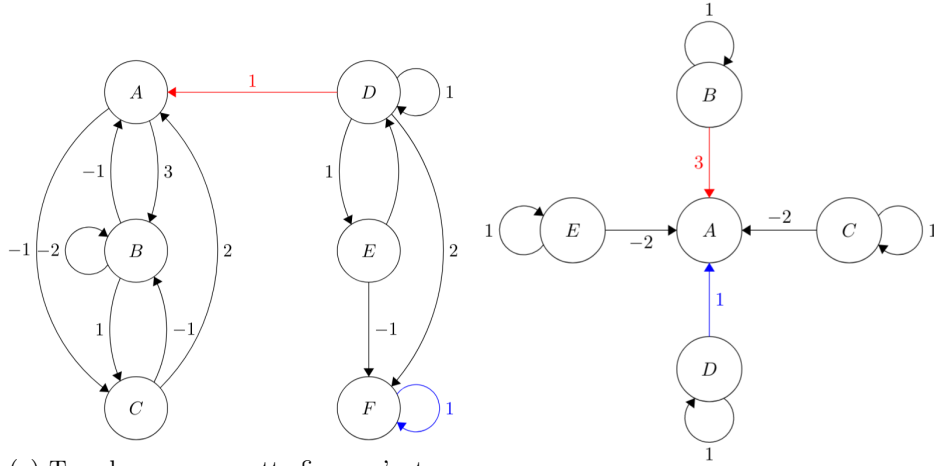
Ainsi, si l'influence d'un chemin est proche de 1, cela signifie que l'état du sommet de départ détermine en grande partie l'état du sommet d'arrivée (après un certain nombre d'étapes). Un chemin d'influence nulle signifie qu'un de ses arcs est de poids nul, donc qu'il n'existe pas dans le réseau.

Après avoir défini l'influence d'un chemin, on peut définir la distance d_{SC} sur les chemins entre deux réseaux ainsi :

$$d_{SC}(g; g') = \sum_{c \in \text{paths}(g) \cap \text{paths}(g')} |I_g(c) - I_{g'}(c)| + \sum_{c \in \text{paths}(g) \setminus \text{paths}(g')} |I_g(c)| + \sum_{c \in \text{paths}(g') \setminus \text{paths}(g)} |I_{g'}(c)| \quad (6)$$

Ou de manière équivalente, en notant K_d le graphe complet à d sommets :

$$d_{SC}(g; g') = \sum_{c \in \text{paths}(K_d)} |I_g(c) - I_{g'}(c)| \quad (7)$$



(a) Tous les arcs sur cette figure n'ont pas la même importance. L'arc DA (en rouge) est le seul à faire communiquer le bloc DEF (bleu) avec le bloc ABC , tandis que l'arc FF (en bleu) n'influe que sur le sommet F , car que l'arc DA ait moins d'importance que celui-ci n'a pas d'arc sortant.

(b) Ici, l'arc BA (en rouge) et l'arc DA (en bleu) ont une position équivalente, mais ils n'ont pas le même poids. On s'attend à ce que l'arc BA car il est de poids plus faible.

Figure 6: Deux illustrations de l'importance d'un arc dans un réseau : à poids équivalent (à gauche) et à position équivalente (à droite)

Notons que d_{SC} ne vérifie pas l'axiome de séparation : en effet, pour $\lambda \in \mathbb{R}_+^*$, $d_{SC}(g, \lambda g) = 0$. Il s'agit donc formellement d'une *pseudométrie* : une fonction qui vérifie tous les critères des distances sauf l'axiome de séparation. Cette dégénérescence est parfaitement justifiée : si l'on considère que les valeurs des poids n'ont de sens que relativement les uns aux autres, il est logique de considérer que deux réseaux identiques à multiplication par un réel strictement positif près sont exactement les mêmes réseaux.

Par la suite, pour simplifier l'écriture, on continuera d'utiliser le terme "distance" en parlant de pseudométries.

3.2.2 Distances dynamiques

De même qu'on ne s'intéressait qu'à la représentation structurelle des réseaux pour calculer les distances structurelles, on ne s'intéressera dans cette section qu'à la représentation dynamique, c'est-à-dire formellement la fonction de succession et informellement le comportement du réseau. On définit la distance dynamique de Hamming et la distance sur les attracteurs.

Distance dynamique de Hamming - HDYN

De même que sur la structure, on peut définir une distance de Hamming sur la dynamique du

réseau. On définit d'abord la distance de Hamming entre deux états avec :

$$\forall s, s' \in \mathbb{S}; H(s, s') = \sum_{i=1}^d |s_i - s'_i|$$

Et on définit la distance dynamique de Hamming entre deux réseaux :

$$d_{DH}(g; g') = \sum_{s \in \mathbb{S}} H(succ_g(s); succ_{g'}(s)) \quad (8)$$

Cette distance représente comme proches deux réseaux où la fonction *succ* ne change que sur peu d'entrées, ce qui est une bonne première approche de la distance entre comportements.

Distance sur les attracteurs - CYCL

Suivant le système étudié, comparer directement les fonctions de succession n'est pas forcément la meilleure représentation. Généralement, l'intégralité de la fonction *succ* n'est pas nécessaire à l'étude du système, on a un surplus d'information. Il suffit souvent de ne connaître que les *attracteurs* et les *bassins d'attraction*. Définissons ces deux notions formellement :

On appelle *attracteur* d'un réseau g une suite d'états $(s_0, s_1, \dots, s_k)$ telle que

$$\forall (0 \leq i \leq k-1), succ_g(s_i) = s_{i+1} \text{ et } succ_g(s_k) = s_0$$

k est la taille de l'attracteur. On appelle *point fixe* un attracteur de taille 1 et *cycle* un attracteur qui n'est pas un point fixe.

On appelle *bassin d'attraction* d'un attracteur $a = (s_0, \dots, s_k)$ (dans un réseau g) l'ensemble des états qui mènent à cet attracteur par applications répétées de la fonction *succ* _{g} :

$$BA(a) = \{s \in \mathbb{S}; \exists n \in \mathbb{N}; succ_g^n(s) = s_0\}$$

De ces définitions découlent les propriétés suivantes pour tout SBN g :

- L'état nul est un point fixe de g . En effet, la définition du comportement des SBN induit $succ_g(0) = 0$.
- Tout état s appartient à un bassin d'attraction. En effet, la suite $(succ_g^n(s))_{n>0}$ est à valeurs dans un ensemble fini, donc est périodique à partir d'un certain rang. La période correspond à l'attracteur.
- L'ensemble des bassins d'attraction de g forme une partition de $\mathbb{S}$. Ce fait découle directement de la propriété précédente.

Bien souvent, il est important de considérer les attracteurs dans l'étude des systèmes, car ils représentent leurs états asymptotiques. L'étude des bassins d'attraction est importante également dans une moindre mesure, car ils témoignent de nombreuses propriétés de l'état asymptotique correspondant du système biologique. Suivant les modèles, un attracteur avec un plus grand bassin d'attraction aura une plus grande stabilité face aux changements d'états, ou une plus grande probabilité d'apparaître.

On cherche donc à construire une distance selon ces contraintes :

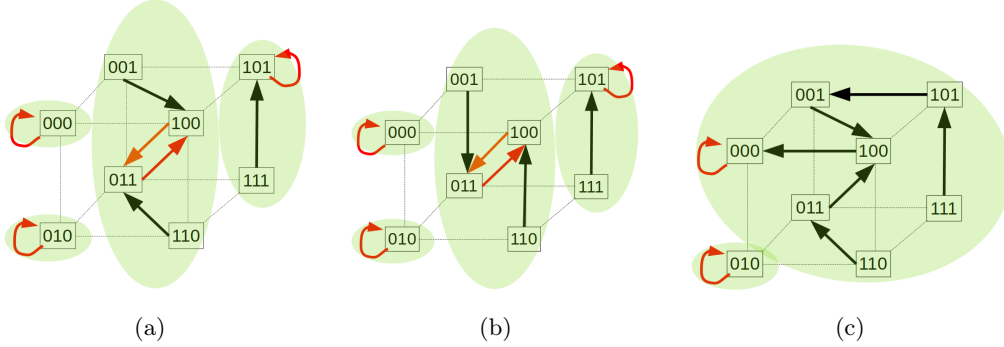


Figure 7: Trois graphes de transition avec différents bassins d'attraction. Selon la distance de Hamming, (a) est plus proche de (c) que de (b), mais les attracteurs et les bassins d'attraction sont plus ressemblants entre (a) et (b) qu'entre (a) et (c). Une distance alternative serait utile pour mesurer ces différences.

- Modifier un bassin d'attraction coûte moins cher que de modifier son attracteur.
- Modifier (ou supprimer) un attracteur à petit bassin d'attraction coûte moins cher que de modifier (ou supprimer) un attracteur avec un grand bassin d'attraction.

Nous allons prendre en compte la taille des bassins d'attraction. Pour cela on effectue un calcul sur chaque état du bassin d'attraction. L'ensemble des bassins d'attraction étant une partition de $\mathbb{S}$, il nous faut finalement faire un calcul sur $\mathbb{S}$.

Il nous faut maintenant introduire une distance entre les attracteurs pour les différencier.

Il n'est pas évident de mesurer une distance entre deux attracteurs de longueur différentes, car une boucle n'a pas de début ni de fin et qu'on compare des objets de tailles différentes.

On va devoir tester tous les débuts de boucle possibles et prendre le minimum, ce qui donne pour deux attracteurs de même taille :

$$d_{CYCLE,1}(\mathcal{C}_1; \mathcal{C}_2) = \min_{1 \leq j \leq |\mathcal{C}_1|} \left(\sum_{i=1}^{|\mathcal{C}_1|} H((\mathcal{C}_1)_{i+j}; (\mathcal{C}_2)_i) \right) \quad (9)$$

où l'on a $(\mathcal{C}_1)_1 = (\mathcal{C}_1)_{|\mathcal{C}_1|+1}$.

Concernant la longueur, on choisit ici d'allonger le cycle le plus court en ajoutant l'état nul autant de fois que nécessaire, ce qui nous donne pour un début de boucle arbitraire si $|\mathcal{C}_1| \geq |\mathcal{C}_2|$:

$$d_{CYCLE,2}(\mathcal{C}_1; \mathcal{C}_2) = \sum_{i=1}^{|\mathcal{C}_2|} H((\mathcal{C}_1)_i; (\mathcal{C}_2)_i) + \sum_{i=|\mathcal{C}_2|+1}^{|\mathcal{C}_1|} H((\mathcal{C}_1)_i; 0) \quad (10)$$

On obtient donc en combinant les deux solutions :

$$d_{CYCLE}(\mathcal{C}_1; \mathcal{C}_2) = \begin{cases} \min_{1 \leq j \leq |\mathcal{C}_1|} \left(\sum_{i=1}^{|\mathcal{C}_2|} H((\mathcal{C}_1)_{i+j}; (\mathcal{C}_2)_i) + \sum_{i=|\mathcal{C}_2|+1}^{|\mathcal{C}_1|} H((\mathcal{C}_1)_{i+j}; 0) \right) & \text{si } |\mathcal{C}_1| \geq |\mathcal{C}_2| \\ d_{CYCLE}(\mathcal{C}_2; \mathcal{C}_1) & \text{sinon} \end{cases} \quad (11)$$

Avec ces deux mesures, on peut enfin définir une distance d_{DA} sur les attracteurs des réseaux. En notant $\lim_g(s)$ l'attracteur dans lequel finit s dans g :

$$d_{DA}(g; g') = \sum_{s \in \mathbb{S}} d_{CYCLE}(\lim_g(s); \lim_{g'}(s)) \quad (12)$$

Cette distance remplit bien nos deux critères de construction : si seul le bassin d'attraction est modifié, seuls les états déplacés produiront une différence, tandis que si l'attracteur change, tous les états du bassin produiront une différence. En outre, si un attracteur est modifié, le coût de la modification est proportionnel à la taille du bassin d'attraction.

Comme on le voit en figure 7, deux comportements ((a) et (b)) peuvent avoir les mêmes attracteurs et les mêmes bassins d'attraction. Ainsi, de même que la distance sur les chemins vue plus tôt, la distance sur les attracteurs ne respecte pas non plus l'axiome de séparation : c'est une pseudométrie. Cela ne cause pas de problème non plus : travailler avec d_{DA} suppose que seuls les attracteurs comptent dans l'étude, il est donc normal de ne pas différencier les comportements sur d'autres critères.

3.2.3 Structure de l'espace des réseaux

Pour pouvoir étudier les concepts évolutifs par la suite, il est nécessaire tout d'abord de se faire une idée de la structure de l'espace des réseaux induite par la distance choisie. Pour cela, nous allons revenir sur les deux distances structurelles définies plus haut.

D'une part, la distance structurelle de Hamming d_{SH} est à valeurs entières, et permet de définir de manière très claire la notion de réseaux voisins : deux réseaux g et g' sont dits *voisins* si $d_{SH}(g, g') = 1$. Cela nous permet également de définir clairement une mutation : la transformation d'un réseau en un de ses voisins. Ainsi, l'espace des réseaux prend la forme d'un graphe non orienté où chaque réseau est connecté à ses voisins.

D'autre part, la distance sur les chemins d_{SC} prend un très grand nombre de valeurs différentes rationnelles, et il est plus ambigu de définir à partir de quelle distance deux réseaux sont voisins. Il est plus cohérent de considérer l'espace des réseaux comme un espace métrique où chaque réseau est bel et bien en relation avec tous les autres.

Ainsi, suivant si d_S est à valeurs discrètes ou continues, on a deux structures bien différentes. J'ai décidé de ne pas choisir une structure ou l'autre, mais d'adapter le système de manière à pouvoir prendre en compte chacun des deux cas. La raison est que les deux structures ont leurs propres avantages et inconvénients. La structure de graphe est plus simple et bien adaptée à une première approche des SBN, mais ne contient en réalité que peu d'informations et peut ne pas traduire une complexité existante ; tandis que la structure d'espace métrique permet de poser un formalisme qui conserve une certaine complexité. En outre, à nouveau, certains systèmes se prêtent mieux à l'étude avec une certaine structure que d'autres. On peut comprendre que des réseaux génétiques tendent à évoluer par petites mutations tandis que les écosystèmes peuvent faire de grands pas d'évolution, chaque espèce évoluant séparément.

Notons que la frontière entre graphe et espace métrique est perméable. D'une part, un graphe peut aussi être vu comme un espace métrique et, d'autre part, si d_S est à valeurs continues, on peut fixer un seuil en-deçà duquel deux réseaux sont considérés comme voisins et obtenir ainsi une structure de graphe.

Du fait de cette distinction, il sera nécessaire pour définir certaines notions, comme celles de chemins dans l'espace des réseaux, de prêter attention à la structure de l'espace dans laquelle on

travaille.

3.3 Formalisation de l'évolution

3.3.1 Robustesse

La plupart des notions qui suivent sont définies à partir de calculs de moyennes ou de sommes sur l'ensemble des réseaux. C'est pour cette raison qu'à partir de maintenant, on mènera l'étude non plus sur $\mathbb{G}$ mais sur $\mathbb{G}/\mathcal{R}$ où $\mathcal{R}$ est une relation d'équivalence quelconque remplissant les conditions énoncées plus haut.

La robustesse d'un système est souvent définie comme sa capacité à conserver son comportement lorsqu'il subit des mutations. On cherche donc à traduire cette notion par une fonction $R : \mathbb{G}/\mathcal{R} \rightarrow [0; 1]$ avec $R(g) = 1$ dans le cas (parfois non atteint) où toutes les mutations conservent le comportement de g . Je donne ici pour illustrer cette notion deux exemples de définitions de la robustesse suivant la structure de l'espace des réseaux.

Structure de graphe : Dans le cas où l'espace des réseaux adopte une structure de graphe, on a vu qu'une mutation se traduit formellement par la transformation d'un réseau en l'un de ses voisins. Pour la conservation du comportement, en l'absence de données supplémentaires sur le système biologique étudié, on fixe arbitrairement un seuil δ en-dessous duquel le comportement est considéré comme étant conservé. On obtient ainsi la formule suivante :

$$R_\delta(g) = \frac{|\{g' \in \text{Voisins}(g); d_D(g, g') < \delta\}|}{|\text{Voisins}(g)|}$$

Structure d'espace métrique : Dans le cas où l'espace des réseaux a une structure d'espace métrique, il est nécessaire de considérer l'intégralité des autres réseaux comme résultant potentiellement de mutations. On pondère chacun des réseaux par sa distance au réseau étudié et par la différence de comportement, pour obtenir la formule suivante :

$$R(g) = \frac{\sum_{g' \in \mathbb{G}/\mathcal{R}} (e^{-d_S(g, g')} \times e^{-d_D(g, g')})}{\sum_{g' \in \mathbb{G}/\mathcal{R}} (e^{-d_S(g, g')})}$$

On a bien ici $R(g) = 1$ dans le cas limite où tous les autres réseaux ont le même comportement que g .

3.3.2 Coût d'un chemin d'évolution, coût d'une évolution

Avant de définir l'évolutivité, il est nécessaire de définir deux notions intermédiaires : le coût d'un chemin d'évolution et le coût d'une évolution. Dans tout ce rapport, on définit formellement un chemin d'évolution comme une suite finie de réseaux $(g_0; \dots; g_k)$ et on appelle *réseau de départ* g_0 et *réseau d'arrivée* g_k . On définit une évolution comme un couple de réseaux $(g \rightarrow g')$, où g est le réseau de départ et g' le réseau d'arrivée. On désigne donc ici par un chemin d'évolution une trajectoire dans l'espace des réseaux, tandis qu'une évolution désigne une transformation d'un réseau en un autre, indépendamment du chemin choisi. Les notions de coût de chemin d'évolution et de coût d'évolution ont pour objet de traduire la difficulté ou la rareté d'une certaine évolution d'un système biologique.

On cherche donc à définir deux fonctions CC (coût d'un chemin) et CE (coût d'une évolution), l'une sur les chemins dans $\mathbb{G}/\mathcal{R}$, l'autre sur $(\mathbb{G}/\mathcal{R})^2$, à valeurs dans $\overline{\mathbb{R}}^*$. On doit avoir en outre $CE(g \rightarrow g') = \infty$ s'il n'y pas de chemin de g à g' .

À nouveau, ces notions ont des définitions très différentes suivant si l'espace des réseaux a une structure de graphe ou d'espace métrique, et il convient de distinguer les deux cas.

Structure de graphe : La notion de coût d'une transformation est complexe et intimement liée au système étudié. C'est pourquoi il vaut mieux sans doute faire le choix de la définition lors du choix du système étudié. Je donnerai cependant ici une définition simple à titre d'exemple du coût d'un chemin d'évolution et d'une évolution.

Dans une structure de graphe, on se restreint aux chemins de voisins en voisins. Un chemin d'évolution est donc une suite finie de réseaux telle que deux réseaux consécutifs sont voisins. Définissons par exemple le coût d'un chemin d'évolution comme étant la longueur du chemin :

$$CC(g_0; \dots; g_k) = k$$

On peut alors définir le coût d'une évolution comme étant le coût minimal d'un chemin allant du réseau de départ au réseau d'arrivée :

$$CE(g \rightarrow g') = \begin{cases} \min_{c \in \text{paths}(g \rightarrow g')} CC(c) & \text{si } \text{paths}(g \rightarrow g') \neq \emptyset \\ +\infty & \text{sinon} \end{cases}$$

Il est important de noter que CC et CE sont deux notions bien distinctes à ne pas fusionner, pour garantir la flexibilité du système formel élaboré. La définition du coût de chemin d'évolution est sujette à beaucoup de variations suivant le système étudié comme expliqué plus haut, tandis que la définition du coût d'évolution fait déjà intervenir le coût des chemins, et donc peut être commune à de nombreux systèmes biologiques.

Structure d'espace métrique : Comme dit précédemment, la structure d'espace métrique est bien plus compliquée à appréhender que celle de graphe. Une piste de travail consisterait à étudier des marches aléatoires sur l'espace des réseaux.

3.3.3 Évolutivité

L'évolutivité est définie comme la capacité d'un système biologique à acquérir de nouveaux comportements avec de petites évolutions. Pour calculer l'évolutivité d'un système, on fera donc un calcul sur l'ensemble des réseaux. Les évolutions faisant varier le comportement doivent être mises en valeur, tandis que les évolutions coûteuses seront. On peut la définir comme cela:

$$E(g) = \text{moyenne}_{g' \in \mathbb{G}/\mathcal{R}} \left(\frac{d_D(g, g')}{CE(g \rightarrow g')} \right)$$

où *moyenne* désigne ici (et par la suite également) la moyenne arithmétique.

On note ici l'importance d'avoir $CE > 0$. Notons que sont prises en compte les évolutions impossibles dans le calcul. Comme on a $CE(g \rightarrow g') = \infty$, le terme correspondant dans le calcul de la moyenne est nul.

3.3.4 Viabilité et score de viabilité

Viabilité: La contrainte de viabilité de l'environnement détermine la survie des systèmes biologiques. Il y a deux manières de présenter cette contrainte. D'une part, on peut choisir une viabilité binaire, pour laquelle un système survit ou ne survit pas. D'autre part, on peut choisir une viabilité graduelle, pour laquelle un système a plus ou moins de chances de survie. Ce cas, plus réaliste mais plus compliqué, correspond à un animal normal et un autre amputé d'une patte. Les deux animaux peuvent vivre et se reproduire, mais le deuxième a toutefois moins de chances de survie, car il y a une probabilité qu'il finisse par se retrouver dans une situation où sa patte en moins lui est fatale.

On appelle donc *viabilité binaire* toute fonction de $\mathbb{G}/\mathcal{R}$ dans $\{0, 1\}$ et *viabilité graduelle* toute fonction de $\mathbb{G}/\mathcal{R}$ dans $[0; 1]$. On appellera *viabilité parfaite* la fonction V_P constante égale à 1 et *viabilité nulle* la fonction V_N constante égale à 0.

Viabilité et évolution Rajouter une contrainte de viabilité modifie les coûts des chemins d'évolution, les coûts d'évolution et l'évolutivité. On notera respectivement CC_V , CE_V et E_V ces quantités modifiées où V est la viabilité. Cette modification est bien entendu différente suivant le modèle. Or, les définitions données jusqu'ici ne supposent pas l'existence d'une contrainte de viabilité, ce qui revient à se placer dans le cas d'une viabilité parfaite. On voudrait donc avoir les propriétés suivantes :

- Les définitions précédentes correspondent à une viabilité parfaite, donc $CC_{V_P} = CC$, $CE_{V_P} = CE$ et $E_{V_P} = E$.
- Rajouter une contrainte de viabilité ne peut qu'augmenter les coûts et entraver l'évolution, donc on a pour toute viabilité V : $CC \leq CC_V$, $CE \leq CE_V$ et $E \geq E_V$.
- Dans le cas d'une viabilité nulle, il n'y a aucune évolution possible, donc : $CC_{V_N} = CE_{V_N} = \infty$ et $E_{V_N} = 0$.

Nos nouvelles définitions devront donc remplir ces critères. Je donne ici un exemple d'une modification prenant en compte la viabilité (en se plaçant dans une structure de graphe). Les définitions qui suivent sont valables pour une viabilité binaire aussi bien qu'une viabilité graduelle :

$$CC_V(g_0; \dots; g_k) = \frac{k}{\prod_{i=0}^{k-1} V(g_i)}$$

$$CE_V(g \rightarrow g') = \min_{c \in \text{paths}(g \rightarrow g')} CC_V(c)$$

$$E_V(g) = \text{moyenne}_{g' \in \mathbb{G}/\mathcal{R}} \left(\frac{d_D(g, g') \times V(g')}{CE_V(g \rightarrow g')} \right)$$

Elles remplissent bien les contraintes demandées plus haut. En outre, l'esprit de la définition de l'évolutivité est préservé : un système est évolutif si des mutations dans sa structure peuvent induire un caractère *héritable* [5]. Un trait "héritable" est un trait qui fait augmenter les chances de survie du système pour une certaine contrainte de viabilité. On ne peut le définir simplement dans le système formel, mais on constate que nos définitions gardent cet esprit : les évolutions vers des systèmes plus viables, donc qui offrent des caractères héréditaires, sont valorisées.

Score de viabilité : On peut enfin définir le score de viabilité. On cherche ici à définir une fonction sur l'ensemble des viabilités qui attribue un score élevé à une viabilité binaire si elle

ne contraint que peu l'évolution des systèmes, et un score faible si de nombreuses évolutions sont impossibles. Dans le cas d'une viabilité graduelle, on aimerait que les chemins maximisant la viabilité de la trajectoire ne fassent que peu de détours par rapport à la viabilité parfaite. Les contraintes à remplir sont donc les suivantes :

- Sc est une fonction de l'ensemble des viabilités dans $[0; 1]$.
- La viabilité parfaite a un score de 1 et la viabilité nulle a un score de 0.

On a déjà défini avec E_V comment une viabilité contraignait l'évolution, il ne nous reste plus qu'à trouver une formule synthétisant ces valeurs. Dans notre description du score de viabilité, on compare les viabilités au cas idéal. Notre formule sera probablement un quotient avec le cas réel au numérateur et le cas idéal au dénominateur. Enfin, pour intégrer sur l'espace des réseaux, nous choisirons une moyenne. Cela nous donne la formule suivante :

$$Sc(V) = \frac{\text{moyenne}_{g \in \mathbb{G}} E_V(g)}{\text{moyenne}_{g \in \mathbb{G}} E(g)}$$

3.4 Paysage des réseaux

En disposant de toutes ces distances, on peut explorer *l'espace des réseaux*, ie l'ensemble des réseaux d'une dimension donnée doté d'une certaine topologie (avec une ou combinaison de plusieurs distances sur les SBNs). Que les distances soient discrètes ou non, nous travaillons sur un espace des réseaux finis (en prenant le quotient $\mathbb{G}/\mathcal{R}$). On peut donc construire un réseau de réseaux, un *métagraphe* qui relie les SBNs selon des conditions choisies par l'utilisateur. Cette représentation permet d'exprimer certains concept d'évolution avec la théorie des graphes dont les outils conceptuels et de programmation sont bien développés.

3.4.1 Métagraphe des SBNs

Un *metagraphe* en dimension d est un graphe ayant pour sommets les n_d SBNs $\{1, 2, \dots, n_d\}$ de la dimension d , reliés selon une ou des *conditions*. Ces conditions peuvent être des conditions de distance, de viabilité... Les arcs peuvent prendre comme poids la distance dynamique entre les deux réseaux qu'ils relient. Si je présente ici les métagraphes que j'ai le plus utilisé dans ce travail, rien n'empêche de les modifier ou d'en créer des nouveaux selon les besoins.

Le métagraphe structurel Le métagraphe le plus simple à construire est celui qui relie les réseaux par proximité structurelle.

Le métagraphe structurel $MG(d_{struct}, \epsilon)$ a pour condition de distance:

$$g \text{ et } g' \text{ sont relis} \Leftrightarrow d_{struct}(g, g') < \epsilon$$

ϵ est le seuil d'acceptabilité pour la distance structurelle. Par défaut, la distance structurelle utilisée est HSTR et les arcs ne sont pas pondérés.

Le métagraphe structure-dynamique Le metagraphe structure-dynamique relie les réseaux proche en structure et en dynamique selon des seuils.

Le métatagraphe structure-dynamique $MG(d_{struct}, d_{dynamique}, \epsilon, \delta)$ a pour condition de distance:

$$g \text{ et } g' \text{ sont relis} \Leftrightarrow (d_{struct}(g, g') < \epsilon) \& (d_{dynamique}(g, g') < \delta)$$

Les arcs peuvent être pondérés par la distance dynamique par exemple: $w_{ij} = d_{dynamique}(i, j)$

Le métatagraphe de viabilité Un métatagraphe de viabilité ne prend pas l'ensemble des n_d SBNs, mais seulement les SBNs considérés "viabiles". Les conditions pour relier les noeuds peuvent ensuite être des conditions de distances structurelles ou structurelles- dynamique.

- Pour une viabilité binaire, on construit un métatagraphe structurel-dynamique $MG(\epsilon, \delta)$ sur le sous-ensemble de réseaux viabiles $\{g_i \text{ tel que } V(g_i) = 1\}$
- Pour une viabilité graduelle, on construit $MG(\epsilon, \delta)$ sur l'ensemble complet des SBNs, en associant à chaque noeud sa viabilité en tant que poids.

On pourrait aussi imaginer une viabilité non pas sur les SBNs (sommets du métatagraphe) mais sur les évolutions (arcs du métatagraphe). Un métatagraphe de viabilité relierait deux réseaux g_i et g_j si l'évolution (directe) entre l'un et l'autre est autorisée. Si le sens d'évolution compte, le métatagraphe est orienté. Les "évolutions autorisées" peuvent prendre plusieurs formes. Voici une liste d'idées:

- Viabilité aléatoire : l'arc ij existe avec une probabilité binaire p (viabilité binaire, avec une loi binomiale). Dans le cas d'une viabilité graduelle, l'arc ij prend un poids w_{ij} dans $[0;1]$ selon une loi de probabilité choisie (loi normale par exemple).
- Selection de caractère(s): On peut retirer un ensemble des noeuds des SBNs possédant une certaine caractéristique interdite. Avec les noeuds restants, on construit un métatagraphe structurel-dynamique (ou structurel simple). Voici des exemples de caractères: présence d'un cycle donné (qui peut représenter un cycle mortel dans un système biologique); l'absence de cycle ou des chaînes acycliques trop longues (le système n'est pas en équilibre); présence d'un seul certain nombre de cycle; un enchainement particulier d'état (qui pourrait être mortel)... En enlevant ces noeuds, le métatagraphe n'est plus forcément connexe: il se peut que les réseaux g_i et g_j ne puissent plus évoluer l'un vers l'autre.

3.4.2 Reformulation de certains concepts avec les métatagraphe

Grace aux métatagraphe, on peut reformuler certains concepts évolutifs avec la théorie des graphes. L'avantage est programmatore: les algorithmes sur les graphes sont bien développés, ce qui permet de faire des analyses facilement et rapidement (ce développement logiciel est détaillé plus tard dans le rapport).

La robustesse peut s'exprimer avec les degrés des noeuds. On construit:

- MG_1 le métatagraphe complet, ie avec tous les voisins structurels:

$$g \text{ et } g' \text{ sont reliés} \Leftrightarrow d_{struct}(g, g') < \epsilon$$

- MG_2 le métatagraphe restreint, ie les voisins structurels proches en dynamique. Ceux sont donc les voisins qui conservent le comportement :

$$g \text{ et } g' \text{ sont reliés} \Leftrightarrow (d_{struct}(g, g') < \epsilon) \& (d_{dynamique}(g, g') < \delta)$$

Le nombre de voisin dans la formule initiale de robustesse peut donc être interprété comme le degré du SBN dans le métagraphe. On peut ainsi reformuler la robustesse d'un SBN:

$$R_\delta(g) = \frac{|\{g' \in Voisins(g); d_D(g, g') < \delta\}|}{|Voisins(g)|} = \frac{deg_{MG_2}(g)}{deg_{MG_1}(g)}$$

Un chemin d'évolution peut être considéré comme un chemin dans un métagraphe construit selon les contraintes correspondantes. En particulier, on peut étudier les plus courts chemins pour identifier une trajectoire "optimale" et le coût d'évolution. On peut donc construire (par exemple), le métagraphe structure- dynamique $MG(\epsilon, \delta)$; le coût d'évolution s'interprète alors comme:

$$CE(g \rightarrow g') = \begin{cases} \text{longueur d'un plus court chemin } g \rightarrow g' \text{ dans } MG \text{ si } paths(g \rightarrow g') \neq \emptyset \\ +\infty \text{ sinon} \end{cases}$$

Ici, on ne prend pour évolutions possibles que des petites mutations structurelles (ϵ) engendrant de petites mutations dynamiques (δ). C'est une hypothèse assez forte, qui exclut les "grands sauts" qui font apparaître de grandes innovations évolutives.

Le coût d'évolution peut être affiné en pondérant les arcs avec une distance dynamique par exemple - une mutation engendrant une grande différence dynamique sera plus lourde en coût d'évolution.

L'évolutivité est ensuite facile à calculer, elle reste identique à sa définition précédente. En pondérant les arcs de MG par les distances dynamiques, le métagraphe suffit à lui seul pour la calculer.

$$E(g_i) = moyenne_{g_j \in \mathbb{G}/\mathcal{R}} \left(\frac{d_D(g_i, g_j)}{CE(g \rightarrow g')} \right) = moyenne_{g_j \in MG} \left(\frac{w_{ij}}{CE(g_i \rightarrow g_j)} \right)$$

Viabilité Pour inclure la viabilité, on peut faire attribuer à chaque SBN g_i une viabilité $V(g_i)$ (un poids $w(g_i)$ sur le noeud g_i). Les chemins sont alors calculés en tenant compte de la viabilité des SBNs rencontrés:

$$CC_V(g_0; \dots; g_k) = \frac{k}{\prod_{i=0}^{k-1} V(g_i)} = \frac{k}{\prod_{i=0}^{k-1} w(g_i)}$$

Si le chemin passe par un noeud non viable ($V(g) = 0$), $CC_V = +\infty$, ce chemin ne peut pas être pris. Le coût d'évolution reste inchangé (longueur du plus court chemin) et l'évolutivité devient:

$$E_V(g_i) = moyenne_{g_j \in \mathbb{G}/\mathcal{R}} \left(\frac{d_D(g_i, g_j) * V(g_i)}{CE(g \rightarrow g')} \right) = moyenne_{g_j \in MG} \left(\frac{w_{ij} * w(g_i)}{CE(g_i \rightarrow g_j)} \right)$$

3.5 Synthèse

3.5.1 Tableau de synthèse sur les distances

Catégorie	Sigle	Distance	Interprétation	Expression
Structure	HSTR	Distance de Hamming structurelle	Différences sur le poids des arcs, ie nombre de mutations entre deux réseaux	$\forall g, g' \in \mathbb{G},$ $HSTR(g, g') = \sum_{i=1}^d \sum_{j=1}^d g_{ij} - g'_{ij} $
	PATH	Distance sur les chemins	Prend en compte l'importance des arcs dans le réseau (leur centralité)	Influence d'un chemin: $I_g(c) = \prod_{a \in arcs(c)} I_g(a)$ Distance entre deux chemins, avec des graphes complets: $d_{SC}(g, g') = \sum_{c \in paths(K_d)} I_g(c) - I_{g'}(c) $
Dynamique	HDYN	Distance de Hamming dynamique	Changement sur les successeurs de chaque état	Distance de Hamming entre deux états: $\forall s, s' \in \mathbb{S}; H(s, s') = \sum_{i=1}^d s_i - s'_i $ Distance dynamique de Hamming entre deux réseaux: $d_{DH}(g, g') = \sum_{s \in \mathbb{S}} H(succ_g(s); succ_{g'}(s))$
	CYCL	Distance sur les cycles	Changement sur les cycles	Distance entre deux cycles: $d_{CYCLE}(\mathcal{C}_1; \mathcal{C}_2) = \begin{cases} \min \left(\sum_{i=1}^{ \mathcal{C}_2 } H((\mathcal{C}_1)_{i+j}; (\mathcal{C}_2)_i) + \sum_{i= \mathcal{C}_2 +1}^{ \mathcal{C}_1 } H((\mathcal{C}_1)_{i+j}; 0) \text{ si } \mathcal{C}_1 \geq \mathcal{C}_2 ; \\ d_{CYCLE}(\mathcal{C}_2; \mathcal{C}_1) \text{ sinon} \end{cases}$ Distance sur les cycles entre deux réseaux: $d_{DA}(g, g') = \sum_{s \in \mathbb{S}} d_{CYCLE}(lim_g(s); lim_{g'}(s))$

PATH est la seule de ces distances à ne pas être à valeurs discrètes.

3.5.2 Evolution

Concept	Interprétation	Définition	Dans les métagraphes
Robustesse	Conservation du comportement malgré la/les mutations(s)	Structure de graphe: $\frac{ \{g' \in Voisins(g); d_D(g, g') < \delta\} }{ Voisins(g) }$ Espace métrique: $\frac{\sum_{g' \in \mathbb{G}/\mathcal{R}} (e^{-d_S(g, g')} \times e^{-d_D(g, g')})}{\sum_{g' \in \mathbb{G}/\mathcal{R}} (e^{-d_S(g, g')})}$	$\frac{deg_{MG_2}(g)}{deg_{MG_1}(g)}$
Chemin d'évolution (coût)		$CC(g_0; \dots; g_k) = k$	Longueur du chemin $g_0, g_1 \dots g_k$
Coût d'une évolution		$CE(g \rightarrow g')$ $= \begin{cases} \min_{c \in paths(g \rightarrow g')} CC(c) \\ \text{si } paths(g \rightarrow g') \neq \emptyset \\ +\infty \text{ sinon} \end{cases}$	$CE(g \rightarrow g')$ $= \begin{cases} \text{longueur d'un plus court chemin} \\ \text{si } paths(g \rightarrow g') \neq \emptyset \\ +\infty \text{ sinon} \end{cases}$
Evolutivité	Capacité à intégrer de nouveaux comportements	$moyenne_{g' \in \mathbb{G}/\mathcal{R}} \left(\frac{d_D(g, g')}{CE(g \rightarrow g')} \right)$	$moyenne_{g_j \in MG} \left(\frac{w_{ij}}{CE(g_i \rightarrow g_j)} \right)$

4 Développement logiciel

Mon stage de 5 mois a consisté à développer des outils numériques pour concrétiser le travail de Rémy D. L'essentiel de mon travail a consisté à implémenter les différentes distances entre SBN, puis à construire un paysage des réseaux avec ces distances. J'ai pu calculer toutes les distances de SBNs deux à deux en dimension 2 et 3. La dernière partie du travail a consisté à analyser l'évolution des SBNs dans en dimension 2 avec la robustesse et l'évolutivité.

4.1 Calculs en Java

Dossier `src>Java>SBAN>src>evolution`

Java est le langage utilisé par l'équipe, notamment Rémi Segretain qui a développé une librairie de calcul parallèle, JPP. C'est donc le langage que j'ai utilisé pour faire les gros calculs de distance, car il y avait déjà des outils adaptés à disposition.

4.1.1 Les classes d'objets

On décrit ici les classes Java créées pour manipuler les SBNs et les distances. Cette description est générale, il faut se reporter aux fichiers sources pour avoir plus de détails (notamment sur les méthodes disponibles d'une classe).

Pour les SBNs

(Dossier `src>Java>SBAN>src>CanonicSbn`)

Les SBNs sont les premiers objets avec lesquels on travaille, et donc que l'on a besoin de définir en Java. Deux sous-classes ont été préalablement créées: *CanonicSbnStruct* et *CanonicSbnDynamic* qui caractérisent les deux aspects fondamentaux des SBN, respectivement la structure et la dynamique. Ces deux classes sont synthétisées dans la classe *CanonicSbn*. Un *CanonicSbn* est donc un objet caractérisé par sa dimension, sa structure et sa dynamique (calculée à partir de sa structure). Plusieurs méthodes sont implémentées pour ces objets, dont la génération du voisinage (par rapport à HSTR) et la liste des cycles présents dans le graphe d'interaction (donné sous forme d'un dictionnaire).

Pour les distances

Les distances sont les objets avec lesquels j'ai le plus travaillé. La classe abstraite *Distance* reprend les caractéristiques essentielles d'une distance: c'est une application $d : \mathbb{G}^2 \rightarrow \mathbb{R}^+$ donc elle a pour attributs deux réseaux $g1$ et $g2$ (sous forme *SanonicSbn*), leur dimension *dim* (qui doit être la même) et le résultat *distance*, sous forme d'un *IntegerParameter* qui doit être positif.

La seule définition de distance ne permet pas de mener des calculs concrets. Il a donc fallu implémenter les distances particulières que nous connaissons, présentées précédemment. Chaque distance est donc codée sous la forme d'une classe fille qui hérite de la classe abstraite *Distance*, ce qui permet d'assurer qu'elle vérifie les condition générales d'une distance (deux réseaux de même dimension, résultat réel positif). Trois ont été implémentées: les distances de Hamming structurelles et dynamiques, et la distance sur les cycles. La distance sur les chemins n'a pas été faite car elle requiert des calculs assez lourds (lister tous les chemins) qui n'ont pas été jugés prioritaires.

Remarque: Il est envisageable de traiter des réseaux de dimensions différentes en "transportant" le petit réseau dans la dimension du grand.

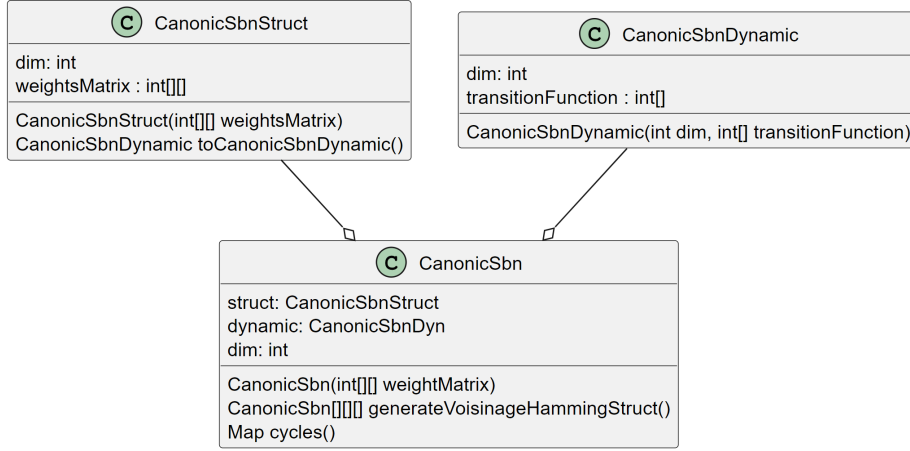


Figure 8: Diagramme UML des classes de SBN

Soit g_1, g_2 deux SNBs de dimensions d_1 et d_2 respectivement, avec $d_2 < d_1$. On définit g'_2 de dimension d_1 de la façon suivante: $sommets(g'_2) = \{1, 2, \dots, d_1\}$ et $arcs(g'_2) = arcs(g_2)$. Ainsi, on peut étudier les distances entre g_1 et g'_2 . On ajoute donc $d_1 - d_2$ sommets déconnectés à g_2 pour le plonger en dimension d_1 .

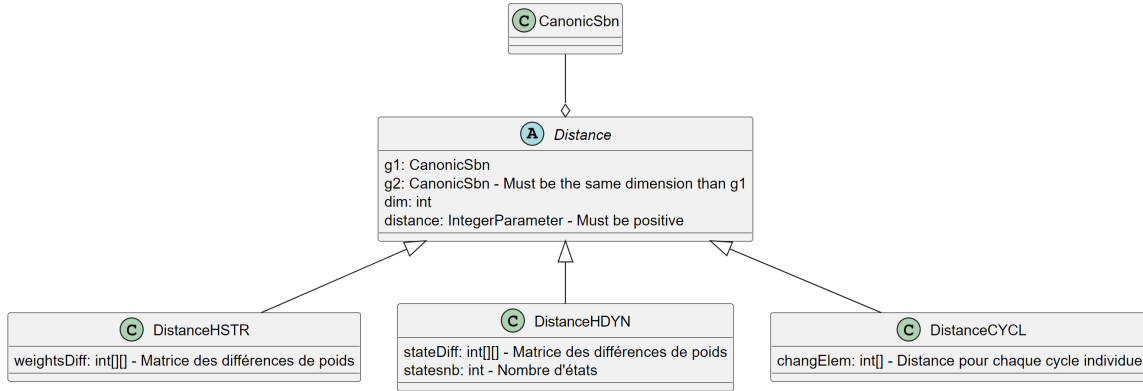


Figure 9: Diagramme UML des classes de distances

4.1.2 La librairie JPP

Lorsqu'on étudie les SBNs, les calculs à faire sont vite importants, en temps et en mémoire (dès la dimension 3, on atteint 31065 réseaux!). Le calcul parallélisé permet de faire plusieurs tâches en même temps (sur différents processeurs), ce qui permet de gagner en temps. Plusieurs ressources sont disponibles pour prendre en main cette librairie:

- La documentation écrite par Rémy Segretain, créateur de JPP

- Un guide d'utilisation que j'ai écrit [donner un PDF]. N'hésitez pas à l'enrichir.

4.1.3 Exploration exhaustive, en d2 et d3

Une grosse partie de mon stage a consisté à calculer toutes les distances (ie, pour toutes les paires possibles) en dimension 2 puis 3. Avec 60 SBNs en dimension 2, puis 31199 en dimension 3, cela en fait du monde! Tellement qu'on ne peut pas échapper au calcul parallèle. [Insérer schémas JPP]
 Cette exploration exhaustive se fait sur un ensemble fini de SBNs: on travaille avec la relation d'équivalence sur les comportements (deux réseaux sont équivalents s'ils ont le même graphe de transition). On ne prend qu'un représentant par classe d'équivalence.

Mes processeurs

Dossier `evolution>Processors`

- **PairGenerator.** $(i0, n) \rightarrow \{(i0, i), \forall i \in [i0 + 1, n]\}$ Génère toutes les paires d'entiers pour $i0$ (plus grandes que $(i0, i0)$). Utilisé pour savoir quelles distances calculer dans le pipeline `DistancesFullPipe`
- **DistancesProcessor** $(id1, id2) \rightarrow ((id1, id2), (hstr, hdyn, cycl))$. Renvoie les distances pour un couple de SBNs. Le processeur se construit en lui donnant la liste des SBNs avec leur identifiant. Ce processeur n'attend qu'à être modifié pour lui ajouter d'autres distances :-)

Mes différentes pipeline

Dossier `evolution>Pipeline`

Attention! Mon code est écrit sous Windows. !! Si vous ne tournez pas sur Windows, il faut changer le processeur `AspTTSBFsGeneratorWindowsProcessor` en `AspTTSBFsGeneratorProcessor`

- **GenerationSbfPipe** Génère les SBFs d'une dimension donnée et les écrit dans un csv. Il y a un processeur à changer selon que l'on est sur Windows ou Linux, bien lire les commentaires du fichier avant de l'utiliser
- **GenerationSbnPipe** Génère les SBNs d'une dimension donnée et les écrit dans un csv. Les SBN sont de la classe `CanonicSbnStruct`.
- **DistancesFullPipe** Calcule les distances pour toutes les paires de SBNs (de type (i,j) avec $i < j$ et les écrit dans un csv.

Le calcul exhaustif des distances

Le pipeline `DistancesFullPipe` permet une exploration exhaustive d'une dimension donnée en calculant toutes les distances, pour chaque paire. Voici comment il marche: [inserer diag JPP]
 Théoriquement, ce calcul peut se faire pour n'importe quelle dimension. En pratique, on ne l'a fait tourner qu'en dimension 2 et 3, le nombre de SBNs devenant trop important au-delà. Ne serait-ce que le fichier de résultat en d3 pèse quasi 10Go, imaginez en dimension 4...

4.2 Analyse de données en Python

Dossier `src>Python>Exploration_dim_2`

Une fois les données générées, elles ont été traitées en Python, tout simplement parce que je maîtrise

bien ce langage, notamment pour le traitement de données. Comme Java, c'est un langage très usuel qui facilite la reprise de mon travail par des personnes suivantes. Le gros du travail a été la réalisation de heatmaps. En dimension 2 puis en dimension 3. Beaucoup d'algorithmes n'ont pas été optimisés et sont donc très lent en dimension 3. Cela peut valoir la peine d'en faire des versions JPP pour améliorer les temps de calculs.

Les grosses fonctions sont écrites dans le fichier **analysis_distances**. les autres fichiers utilisent ces fonctions pour des applications différentes.

4.2.1 Generation de heatmaps

Fichier **heatmaps_plot**

Les données calculées en Java sont récupérées en Python via les csv (d'où l'importance de fichiers et entêtes bien nommés!). Les distances sont écrites d'abord dans un dataframe (pandas) puis dans des matrices. Pour chaque distance, une matrice est créée telle que $m[i][j] = m[j][i] = d(i, j)$. Les matrices sont donc entièrement remplies et symétriques, alors que les fichiers csv ne comportent pas les doublons $d(j, i)$ et les éléments nuls $d(i, i)$ par gain d'espace et de temps. Cependant, en l'état, les résultats ne sont pas ordonnés de façon intéressantes. L'idée est de les réorganiser pour faire apparaître des clusters de proximité. Pour cela, on calcule un dendrogramme, selon une distance de référence (HSTR par défaut). Une fois le dendrogramme calculé, toutes les matrices de distances sont réorganisées selon ce dernier. Ainsi, pour toutes les distances, les paires de SBNs sont dans le même ordre, selon une distance de référence. En dimension 3, la matrice est bien trop grosse pour être traitée et affichée. Les données sont donc agrégées par blocs de taille fixée par l'utilisateur.

- Récupération des données à partir des csv générés en Java, dans un dataframe
- Génération des matrices de distances
- Calcul du dendrogramme avec la matrice de la distance de référence
- Réarrangement des autres matrices selon le dendrogramme obtenus
- Moyennage par blocs de taille **taillebloc** afin de réduire la matrice si nécessaire (d3)
- Affichage sous forme de heatmap

Le moyennage est paramétrisé selon **taillebloc**.

- Si **taillebloc** = 1, la matrice originale est renvoyée.
- Si **nbsbn % taillebloc != 0**, le dernier bloc est partiellement rempli.

4.2.2 Correlation

Fichier **correlations**

Afin d'analyser un peu plus finement les relations entre les distances, les corrélations sont calculées grâce au fichier **correlations**. Il s'agit d'une corrélation linéaire. La fonction **correlation_btw_distances** est écrite dans le fichier **analysis_distances** en utilisant *stats.pearsonr*. Le corrélogramme est ensuite visible sous forme de heatmap. C'est un algorithme calculatoire qui pourrait être parallélisé en Java (la dimension 3 est très très longue à calculer).

4.2.3 Metagraphe

Fichier `metagraphe`

Les métagraphes sont construits avec la librairie `networkx` (`nx`, cf documentation officielle si vous ne connaissez pas). La fonction `build_graph` construit un graphe

- Avec n_d noeuds, labelés de 0 à $n_d - 1$ (n_d nombre de sbns en dimension d)
- Les arcs sont créés à partir des matrices de distances structurelle et dynamique et leur seuils. Si $d_{struct}(i, j) \leq \epsilon$ & $d_{dynamique}(i, j) \leq \delta$, l'arc (i, j) est créé.

Les poids des arcs peuvent être changés dans le code selon les besoins.

La visualisation est faite en combinant `networkx` et `matplotlib`, qui peut être un peu touchy. La commande de base est `nx.draw(G, kwargs)`. Les kwargs sont très variés et permettent de personnaliser le graphique comme en `plt`.

Disposition des noeuds: Pour comparer des graphes (deux MG avec des seuils différents par exemple), on souhaite que les noeuds soient disposés de la même façon pour tous. Par défaut, les sommets sont disposés de façon à optimiser la disposition, donc elle change d'un graphe à l'autre. Pour fixer cette disposition, il faut jouer avec le `layout`. Pour cela, il faut utiliser le layout fixe `layout2` en dimension 2 dans le fichier `mg_layout`. La fonction `compare_metagraphe_plot` permet de comparer deux métagraphes structures dynamiques avec la même disposition des noeuds et les arcs non communs en rouge.

```
#il faut donner exactement le même layout aux deux graphes
nx.draw(G1, pos = layout2)
nx.draw(G2, pos = layout2)
plt.show()
```

4.2.4 Robustesse et évolutivité

Fichier `robustesse`.

Calculs de robustesse à l'aide des métagraphes (degré des noeuds). Il y a plusieurs fonctions dérivée: robustesse d'un seul noeud, robustesse d'un métagraphe (matrice des robustesses de chaque noeud), robustesse en faisant varier δ , robustesse moyenne selon ϵ et δ .

4.2.5 Trouver un chemin

Fichier `find_path`.

Ce fichier permet de trouver un chemin de g_1 à g_2 avec les données exhaustives de la dimension 2. Ce fichier est obsolète car il n'utilise pas les métagraphes et les fonctions ne marchent pas bien. Libre à vous de faire une version plus aboutie. Voici une proposition d'algorithme avec un métagraphe:

- Construire le métagraphe MG avec les conditions voulues
- Trouver un chemin de g_1 à g_2 en tant que noeuds dans MG , avec un algorithme de chemin (par exemple `shortest_path` de `networkx` calcule tous les chemins les plus courts).

Il pourrait être intéressant d'afficher un chemin sur un métagraphe (en mettant les arcs empruntés en gras par exemple). On pourrait ensuite afficher ce chemin selon différentes conditions (sur ϵ , δ ou condition de viabilité) pour voir comment changent les chemins d'évolution avec ces contraintes.

5 Résultats et analyse

5.1 Listes de SBNs et distances

Dossier `pipelineWorkingFolder>distances_data`.

Les CSV produits par le JPP `DistancesFullPipe` sont enregistrés dans ce dossier. Ils sont écrits avec l'entête suivant: `id_1`, `id_2`, `HSTR`, `HDYN`, `CYCL`

En dimension 3, les SBNs ont été regroupés par blocs (de 100), dans le fichier `distance_dim3_blocks_size_100.csv`.

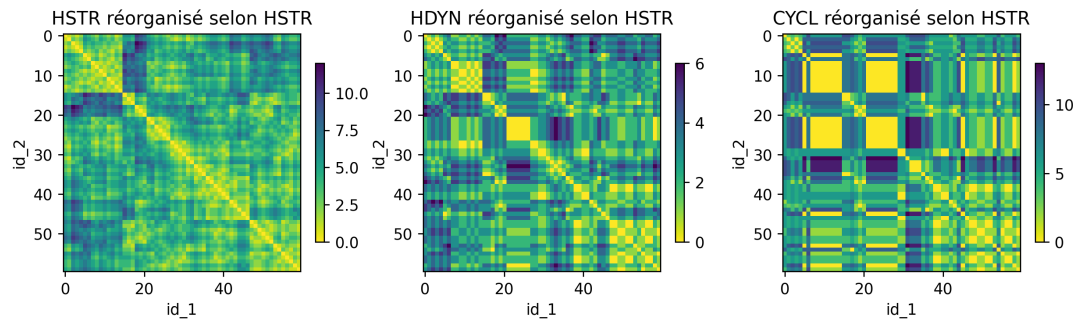
Voici quelques analyses basiques que l'on peut déjà faire:

Dimension	Distance	Min	Moyenne	Max
d2	HSTR	0	4.33	12
	HDYN	0	2.52	6
	CYCL	0	5.23	13
d3	HSTR	0	11.91	30
	HDYN	0	8.83	21
	CYCL	0	21.85	89

5.2 Les heatmaps

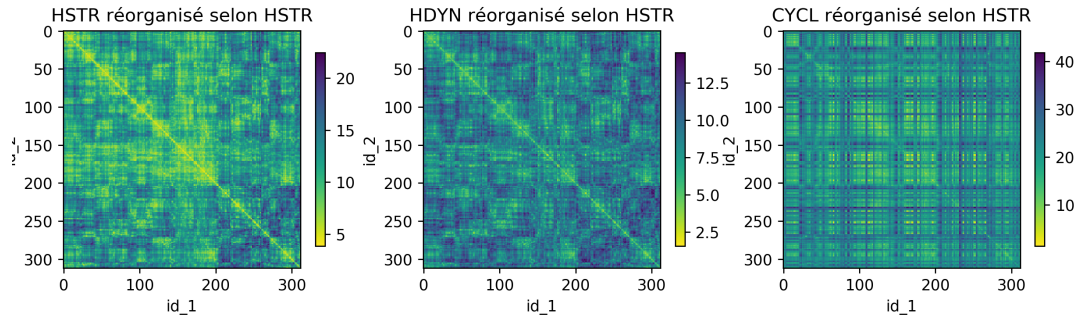
Le pixel (i, j) représente la valeur de $d(i, j)$. Les matrices ont été réarrangées par dendogramme selon HSTR. Le réarrangement par dendogramme selon HSTR rapproche de la diagonale les paires de réseaux proches selon HSTR, pour toutes les matrices (HSTR elle-même mais aussi HDYN et CYCL). On remarque une certaine corrélation visuelle entre les heatmaps d'une même dimension, soulignant qualitativement la corrélation entre les distances, qui a aussi été calculée quantitativement. Ces heatmaps offrent une première représentation visuelle de l'espace des réseaux.

Distance de référence: HSTR



(a) Heatmaps en dimension 2

Distance de référence: HSTR



(b) Heatmaps en dimension 3, blocs de taille 100

Figure 10: Heatmaps

5.3 Correlations

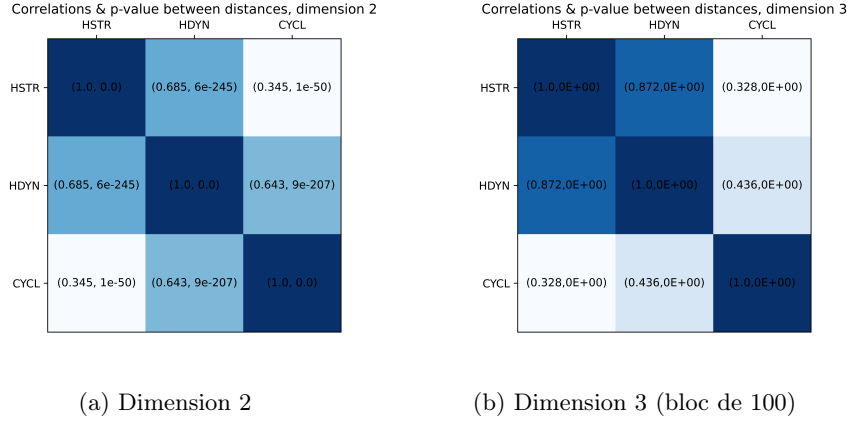


Figure 11: Corrélogrammes

En dimension 2, les trois distances sont très corrélées, avec des p-values quasi nulles. On peut l'interpréter comme: " Une petite mutation structurelle engendre une petite mutation dynamique (avec une très haute probabilité)". La distance CYCL sur les cycles est plus corrélée à HDYN qu'à HSTR: la ressemblance des cycles tient plus à une ressemblance dynamique (sur l'enchaînement d'état) qu'à une ressemblance structurelle.

En dimension 3 Les SBNs étant trop nombreux en dimension 3, ils ont été regroupés par blocs de 100 pour accélérer les calculs (à partir du dendrogramme selon HSTR). Les distances restent très corrélées avec de faibles p-values. Par rapport à la dimension 2, on remarque que HDYN est encore plus corrélée à HSTR, alors que CYCL est un peu moins corrélée au deux autres, surtout HDYN.

5.4 Metagraphe

Les métagraphes permettent de visualiser l'espace des réseaux doté d'une topologie (ici: la distance structurelle à gauche, et les distances structurelle et dynamique à droite). En donnant le même layout à plusieurs métagraphes, les sommets sont positionnés de la même façon, ce qui permet de comparer les différents cas. La figure 12 représente le métagraphe complet pour $\epsilon = 1$ en dimension 2. Ainsi, deux sbns sont reliés s'ils sont voisins directs (distants d'une seule mutation structurelle). Cela nous permet d'avoir une première vision de ce paysage des SBNs.

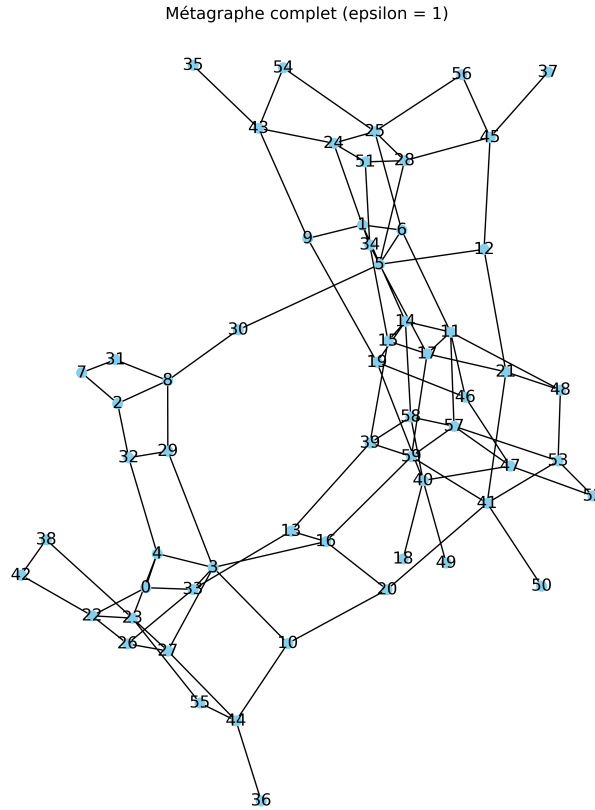
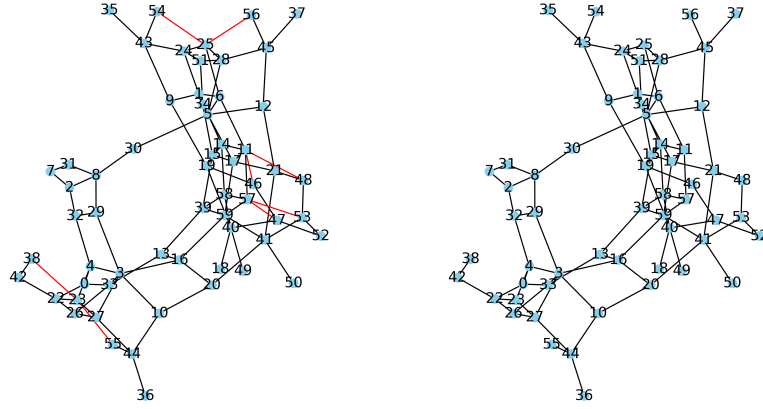


Figure 12: Metagraphe complet MG(epsilon=1), dimension 2

La figure 13 compare ce métagraphe complet avec des métagraphes restreints, pour $\delta = 1$ (13a) et $\delta = 0$ (13b). En rouge sont mis en valeur les arcs retirés du métagraphe complet lorsqu'on passe au restreint. Ainsi, la figure 13a permet d'identifier les mutations engendrant des sauts de comportement. Les SBNs 25 et 54 (en haut) sont distincts d'une seule mutation (structurelle) mais sont assez éloignés en dynamique (de 2 ou plus selon HDYN, ils ne sont plus reliés dans le métagraphe restreint). La mutation 25 - 54 engendre donc un saut de comportement notable. Le métagraphe restreint à $\delta = 0$ (13b) permet d'identifier des clusters de SBNs qui conservent exactement le comportement.

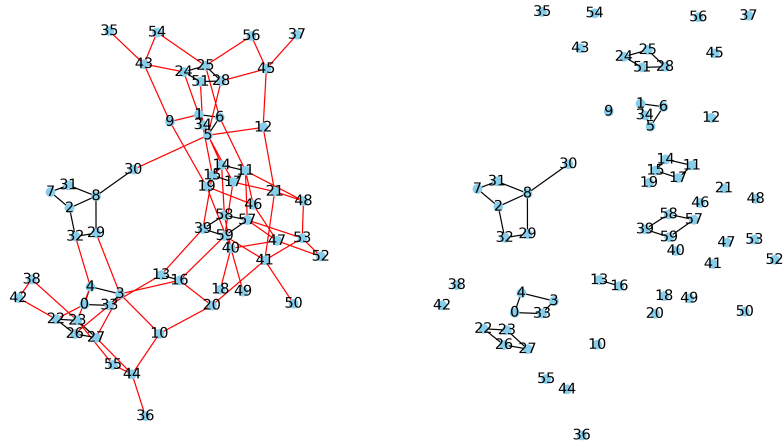
Par défaut, la distance dynamique utilisée est HDYN pour la suite du document.

Métagraphe complet (epsilon =1) et métagraphe restreint (epsilon = 1, delta = 1)



(a) Metagraphe complet (epsilon) et restreint (epsilon, delta), epsilon = 1, delta = 1

Métagraphe complet (epsilon =1) et métagraphe restreint (epsilon = 1, delta = 0)



(b) Metagraphe complet (epsilon) et restreint (epsilon, delta), epsilon = 1, delta = 0

Figure 13: Comparaison de métagraphe complets et restreints en dimension 2

5.5 Robustesse

La robustesse est une première mesure d'évolution que j'ai étudiée dans le paysage des réseaux. La robustesse est un indice dans $[0; 1]$ mesurant la proportion de voisins gardant le même comportement que le réseau étudié. $R(g) = 1$ signifie que tous les voisins de g ont le même comportement que ce dernier; à l'inverse, $R(g) = 0$ montre que le comportement de g n'est pas conservé dans son voisinage.

J'ai d'abord calculé la robustesse de chaque noeud pour un ϵ et un δ fixé. Voici ce que l'on obtient pour $\epsilon = 1$ et $\delta = 0$, pour $d_{dynamique} = HDYN$:

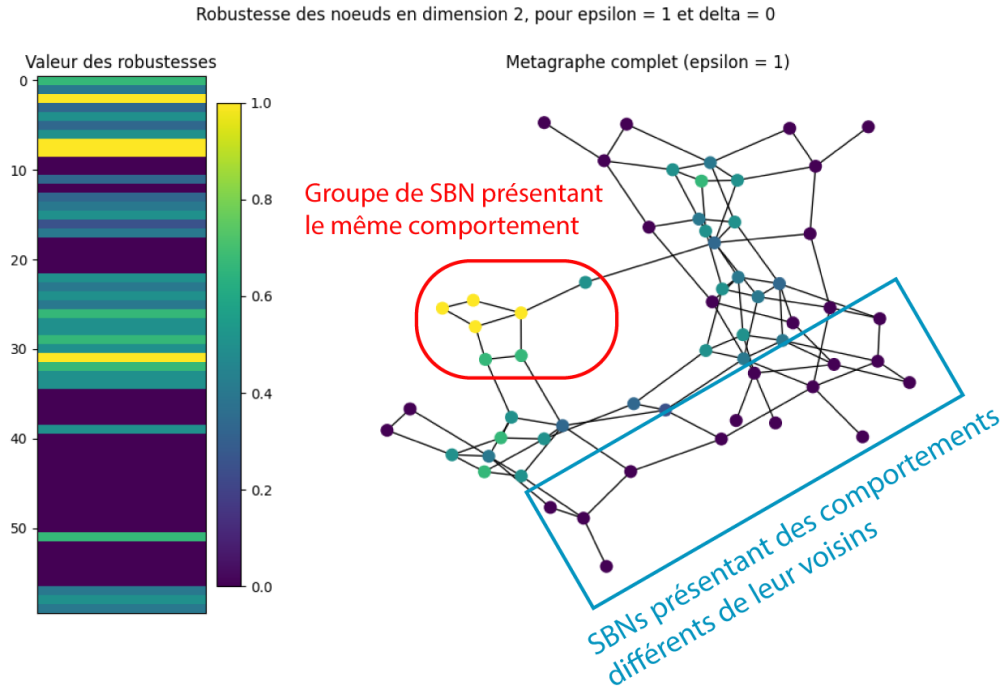


Figure 14: Robustesse de chaque sbn avec $\epsilon=1$ et $\delta=0$ en dimension 2 (HSTR - HDYN)

Ce couple $(\epsilon, \delta) = (1; 0)$ est intéressant car on considère les voisins directs, dont ceux qui conservent exactement le comportement. En jaune sont les SBNs dont tous les voisins conservent exactement le comportement ($R = 1$); en violet ceux dont tous les voisins diffèrent ($R = 0$). Entre ces deux extrêmes se trouvent les variations de bleu vers le vert. Dans cet exemple, on identifie "un cluster" de SBNs partageant exactement le même comportement: les 4 SBNs jaunes et leurs voisins directs. Le comportement qu'ils partagent est donc plus résilient que d'autres car il persiste malgré quelques mutations. On note aussi sur l'extérieur des SBNs non robustes (violet): une mutation sur ces systèmes entraîne un changement de comportement.

Après avoir calculé les robustesses individuelles, j'en ai fait la moyenne. J'ai fait varier cette robustesse moyenne en fonction de ϵ et δ , puis en faisant le calcul pour CYCL.

- $\epsilon = 1$ correspond au cas où les voisins considérés sont les voisins directs, obtenus avec une seule mutation structurelle.
- $\delta = 0$ correspond aux voisins présentant exactement le même comportement.
- ϵ varie jusqu'à 12 car en dimension 2, $HSTR_{max} = 12$; et δ jusqu'à 6 car $HDYN_{max} = 6$. Pour CYCL, $CYCL_{max} = 13$ donc δ va jusqu'à 13.
- Pour HDYN, on remarque que pour $\delta > \epsilon$, $R_{moyenne} = 1$. Lorsque la robustesse moyenne est de 1, tous les sbns sont parfaitement robustes ($R = 1$ pour tous). C'est à dire que tous les voisins structurels sont δ -proches du réseau d'origine. On peut en déduire que ϵ mutations structurelles engendrent au maximum une différence dynamique de $\delta = \epsilon + 1$. Pour CYCL, une mutation structurelle engendre beaucoup plus de degrés différent de distance sur les cycles.

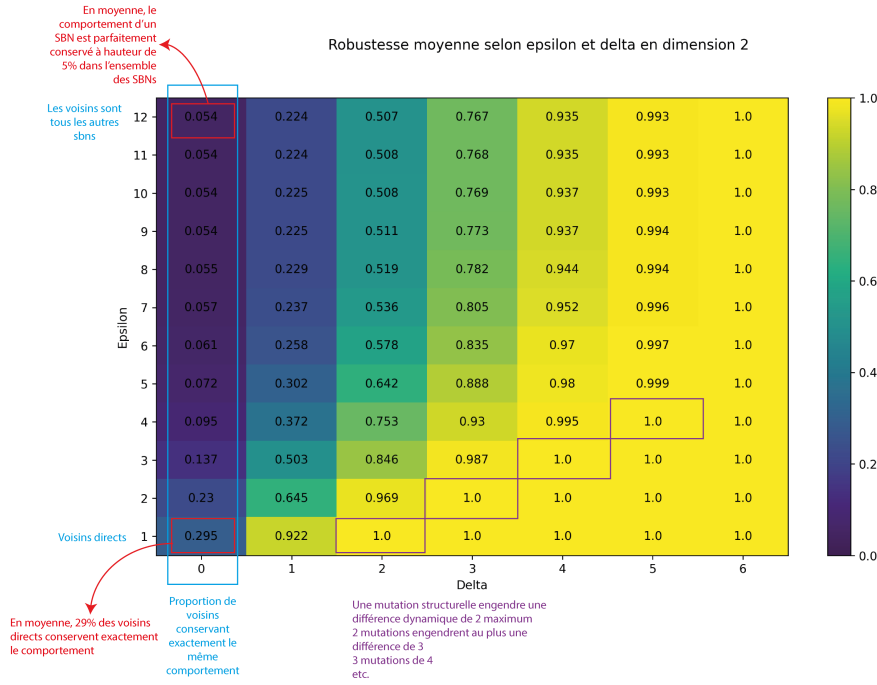


Figure 15: Robustesse moyenne en fonction d' ϵ et δ en dimension 2 (HSTR - HDYN)

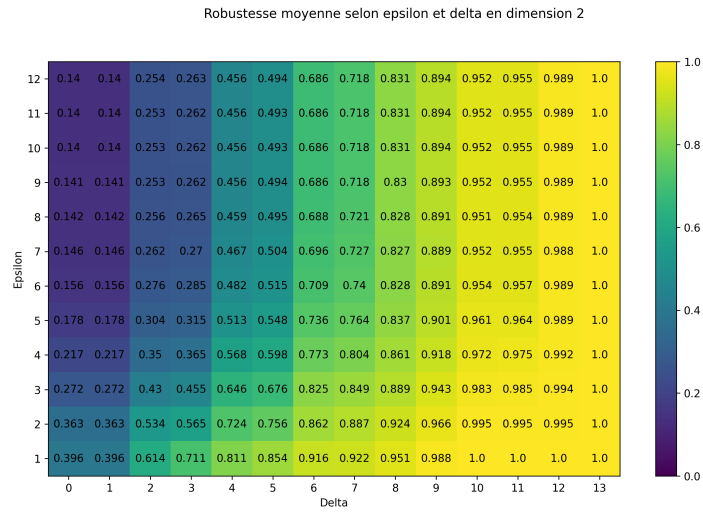


Figure 16: Robustesse moyenne en fonction d' ϵ et δ en dimension 2 (HSTR - CYCL)

5.6 Evolutivité

On obtient le même type de visualisation que pour la robustesse. On peut afficher l'évolutivité de chaque noeud dans le métagraphe, et même la comparer avec la robustesse pour la même situation (mêmes ϵ et δ).

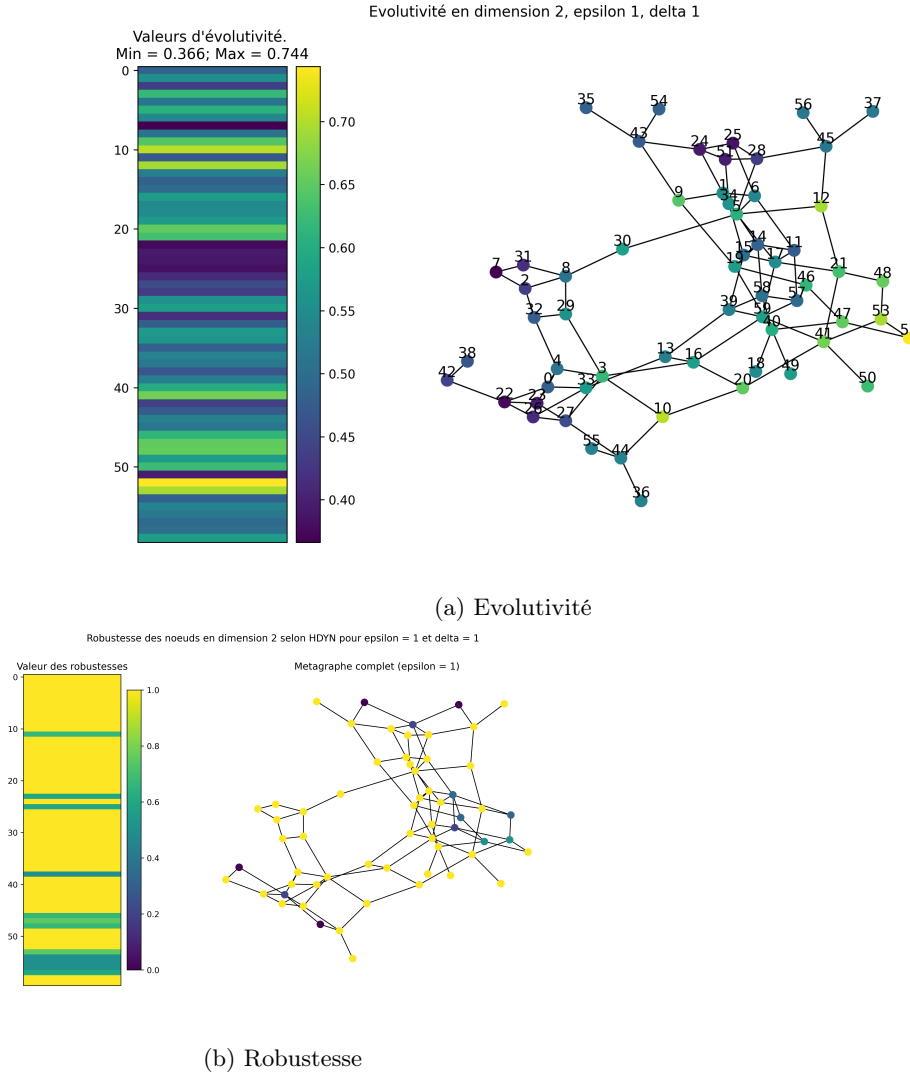


Figure 17: epsilon = 1, delta = 1

L'évolutivité est affichée sur le métagraphe restreint, alors que la robustesse est mise sur le métagraphe complet. L'évolutivité ne prend en compte que les voisins structurels *et dynamiques* (vérifiant les deux seuils); la robustesse prend en compte tous les voisins structurels (vérifiant seulement le

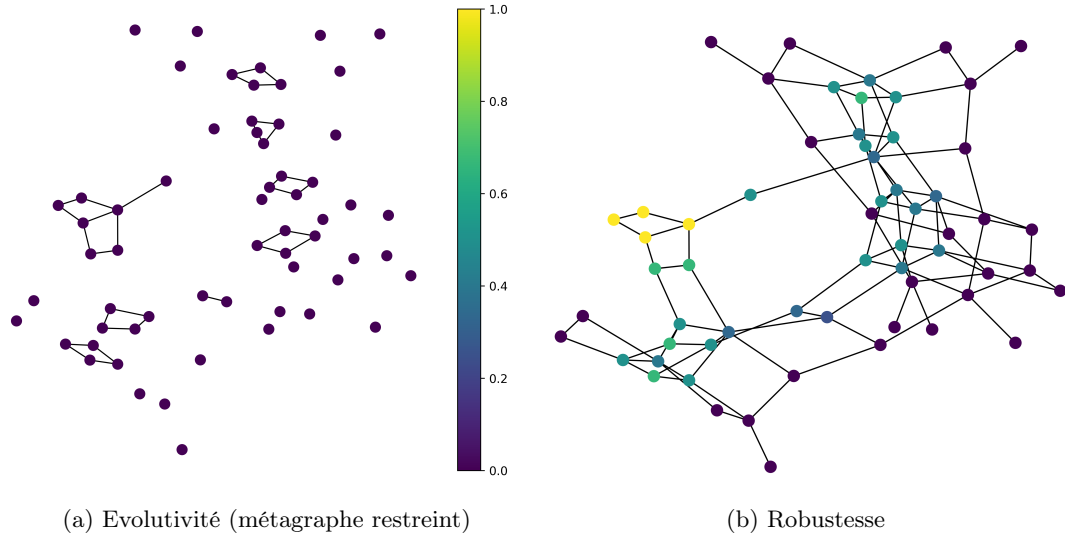


Figure 18: epsilon = 1, delta = 0

seuil structurel) car elle compare le nombre de voisins proches en dynamique par rapport au nombre de voisin total. Attention, il y a quelques problèmes d’affichage sur les colormaps pour les évolutivités; le cas $\delta = 0$ ne s’affiche pas bien. Voici quelques analyses que l’on peut faire sur ces comparatifs:

- Dans la figure 17 sont reliés les voisins directs ($\epsilon = 1$). L’évolutivité et la robustesse sont calculées avec $\delta = 1$, donc en n’autorisant que des petits sauts de comportements. Pour une seule mutation structurelle, la différence dynamique est au plus de 2 ($\epsilon = 1 \implies HDYN(g, g') \leq 2$). Ainsi lorsqu’on prend $\delta = 1$, le seuil de tolérance en dynamique est assez élevé. Les noeuds sont donc facilement considéré comme robuste, ce qui explique qu’un grand nombre d’entre eux soient en jaune (parfaitement robustes). Dans cette situation intermédiaire, l’évolutivité est assez variable.
- Dans le cas $\delta = 0$ (figure 18a), aucun SBN n’est évolutif car on regarde le cas où seuls les évolutions conservant exactement le comportement sont autorisées.
- Comparons les cas intermédiaires: la robustesse pour $\delta = 0$ (figure 18b) et l’évolutivité pour $\delta = 1$ (figure 17a). On repère le cluster parfaitement robuste, déjà identifié dans la figure 14 (sommets jaunes). Ces mêmes noeuds semblent assez peu évolutifs (tirant vers le violet pour l’évolutivité): leurs voisins ayant exactement le même comportement, ils ont besoin plus d’étapes pour acquérir un nouveau comportement. A l’inverse, les SBNs peu robustes sur la diagonale basse (cf 14 encore une fois) sont assez évolutifs: leurs voisins conservant peu leur comportement, ils ont plus facilement accès à de nouveaux comportements. Il semblerait donc que la robustesse et l’évolutivité soient inversement proportionnelles, quelque chose comme $evolutivité = 1 - robustesse$ pour faire grossier. Cette relation est certainement plus subtile car selon [2], robustesse et évolutivité ne s’oppose pas nécessairement. Il faudrait donc faire des calculs de corrélations et des analyses plus poussées pour en avoir le coeur net.

6 Conclusion

Ce stage a permis de visualiser l'espace des réseaux, ce qui est une bonne avancée pour le domaine! La théorisation de Rémy a pu être testée sur de petites dimensions, et de premières analyses ont pu être faites. On remarque tout de suite la corrélation entre distance structurelle et distance dynamique, ce qui fait sens. L'évolution dans le paysage a pu être esquissée avec la robustesse et l'évolutivité, que l'on peut interpréter par zones du métagraphes.

Voici quelques pistes de recherches pour approfondir ce travail:

- Essayer de faire ces analyses en dimensions plus grandes, en faisant du séquençage par exemple.
- Réfléchir sur le lien entre robustesse et évolutivité, telles qu'elles sont données dans ce travail (et dans [1]).
- Faire ces analyses avec d'autres distances (CYCL ou PATH). L'étude des cycles peut être très intéressante, notamment pour la dualité robustesse/ évolutivité (un SBN pourrait garder ses cycles tout en acquérant de nouveaux). PATH n'a pas été utilisée car trop lourde en calculs, mais cela peut valoir la peine d'essayer.
- Appliquer des contraintes de viabilité, ce qui n'a pas du tout été fait dans ce travail. On pourrait alors faire les mêmes analyses et comparer avec la situation sans contrainte; il serait aussi intéressant de regarder quels chemins d'évolution s'en trouvent modifiés.

7 Quelques conventions de travail

Mon stage a permis, à moi mais aussi à l'équipe, d'établir des conventions claires et fixées pour tout le monde, afin d'éviter certaines erreurs ...

7.1 Ecriture binaire

Point sensible! On a vite fait de s'inverser dans le sens d'écriture des nombres binaires (pour les états des SBNs). Voici la convention adoptée par tout le monde. Les états en binaire sont lus "par les humains" dans le même sens qu'un nombre décimal, c'est à dire les grandes puissances à gauche:

$$2^k \mid 2^{k-1} \mid \dots \mid 2^1 \mid 2^0$$

Mais attention! Lorsqu'un nombre binaire est écrit dans un tableau (Python, Java...), c'est inversé car les chiffres du nombres binaires sont écrits selon l'index du tableau (l'index correspond à la puissance de 2 en question).

Index	0	1	...	k-1	k
	2^0	2^1	...	2^{k-1}	2^k

Donc par exemple, pour écrire 3 en 4 bits, cela donne:

Décimal	Pour les humains	Dans un tableau (Python, Java...)
3	011	[1,1,0]

7.2 Dénominations

7.2.1 Pour ne pas se faire de noeuds au cerveau... Désignation des noeuds

Etant donné qu'un SBN est caractérisé par deux graphes (le graphe de structure et le graphe de transition), il ne faut pas confondre les noeuds de l'un et l'autre. On les désigne donc différemment.

g_{ij}	Noeud i à j
<i>sommet</i>	Noeud du graphe de structure (un gène dans un réseaux de régulation génétique par exemple).
<i>état</i>	Noeud du graphe d'interaction. C'est donc un nombre binaire qui représente les valeurs prises par chaque sommet à un instant donné. Par exemple, dans un réseau de régulation à 3 gènes, l'état 000 représente la situation où les 3 gènes sont désactivés.

7.2.2 Sigle des distances

Les distances sur les réseaux sont nommées par sigles de 4 lettres. Ces sigles sont utilisés pour nommer les fichiers, nommer les colonnes de CSV dans mon travail... Les sigles ont la même longueur afin d'être identifiés facilement (lorsqu'on lit le nom de fichier CSV par exemple). Je recommande de conserver ces conventions pour pouvoir facilement réutiliser le travail des uns et des autres. Si une nouvelle distance vient à être utilisée, créez un sigle pour celle-ci (4 lettres capitales) pour l'intégrer facilement.

Distance de Hamming structurelle	HSTR
Distance de Hamming dynamique	HDYN
Distance sur les cycles	CYCL
Distance sur les chemins	PATH

7.2.3 Nom des fichiers

J'ai fait de nombreux CSV pour stocker les distances calculées en JPP. Voici comment les colonnes ont été nommées: `id_1`, `id_2`, `DIST_1`, `DIST_2`

Par exemple, le fichier `distances_dim2.csv` a pour entête `id_1`, `id_2`, `HSTR`, `HDYN`, `CYCL`.

Bien utiliser des dénominations normées permet de récupérer les données beaucoup plus facilement, notamment pour les distances.

7.2.4 Indexation des SBNs

En Java (avec le JPP), les SBNs sont indexés à partir de 1. En python, ils sont indexés à partir de 0 (car tout commence à 0 en Python). La conversion se fait de la génération des matrices, dans la fonction `matrix_distance` dans `analyse_distance.py`. C'est un peu relou, désolée ... Il peut être intéressant de refaire les calculs de jpp avec un index commençant à 0. cette indexation se change dans les `sbngeneratorjob` des pipelines. si vous faites cela, veuillez à bien changer la fonction `matrix_distance` pour que tout soit cohérent.

7.3 Mémo

7.4 Classe d'équivalence de SBNs

Dans ce travail, l'espace des réseaux a été traité comme un ensemble fini avec la relation d'équivalence suivante:

g et g' sont équivalents $\iff g$ et g' ont le même comportement, ie le même graphe de transitions

7.5 Statistiques utiles

Quelques statistiques utiles en dimensions 2 et 3: (pour plus de détails, cf HDR Nico)

Statistiques	d_2	d_3
Nombre de SBNs	60	31.199
Nombre de couples (i, j) avec $i < j$ (sans doublons)	1.770	486.673.201
HSTR max	12	30
HDYN max	6	21

References

- [1] Rémy Défossez. Le millefeuille évolutionnaire, 2021.
- [2] Adami et al. Evolution of biological complexity. 2000.
- [3] Nicolas Glade. Le rare, le faible et l’amorphe dans les systèmes naturels. *HDR en cours de préparation*, 2021.
- [4] Elio Nushi, Victor-Bogdan Popescu, Jose Angel Sanchez Martin, Sergiu Ivanov, Eugen Czeizler, and Ion Petre. Network modeling methods for precision medicine, 2021.
- [5] Andreas Wagner. Robustness and evolvability: a paradox resolved. *Proc. R. Soc. B*, 275:91–100, 2008.